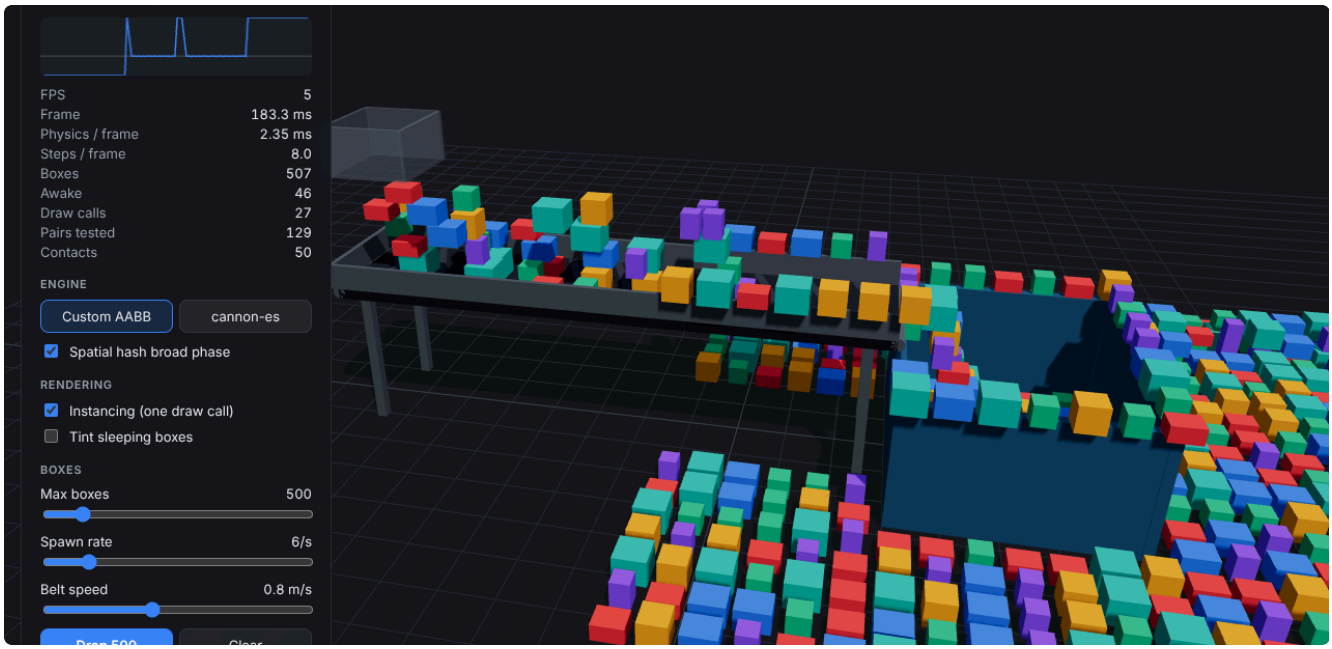
VENTION · PORTFOLIO WALKTHROUGH

Snack 3, Line by Line

Physics in the browser: a fixed-step engine written from scratch, the same scene on cannon-es, and the optimisations that keep thousands of boxes at 60 fps, every file explained.



3 · Conveyor + Physics

Projects in `Portfolio/vention/` · Code quoted directly from the repository, with file and line numbers

Contents

How to read this book 3

Chapter 3 · Conveyor + Physics 4

3.1 What a physics engine does in one step 5

3.2 Time: fixed steps and interpolation 5

3.3 Finding pairs: the broad phase 6

3.4 Contacts and impulses 7

3.5 Stacks, sleeping and islands 8

3.6 Performance in a browser 8

Line by line 9

The work cell as data 9

The fixed-timestep loop 12

Object pooling 14

One interface, two engines 16

The spatial hash 18

The custom engine 22

The cannon-es adapter 40

Spawning and recycling 46

Drawing thousands of boxes 50

The conveyor and bin 54

The frame loop 57

The frame-time graph 61

Interview questions 63

Exercises 65

Appendix A · Running the projects 66

Appendix B · Glossary 67

BEFORE YOU START

How to read this book

This walkthrough covers Snack 3 on its own. It's also Chapter 3 of the combined book, *Five Snacks, Line by Line*, where every chapter has the same five parts:

1. **Concepts.** The ideas the code relies on, with diagrams, before any code.
2. **Line by line.** Every source file that matters, quoted from the repository in order, with each chunk explained. The code is sliced from the real files, so the line numbers match what you'll see in your editor.
3. **Going deeper.** Boxes inside the line-by-line parts that step back to what a production system, a full engine or a large team would do differently.
4. **Interview questions.** Questions this project invites, with model answers grounded in the code.
5. **Exercises.** Extensions to try, each with a hint.

Two reading paths work well. To prepare for a conversation about the project, read the concepts and the questions. To understand or extend the code, read the line-by-line parts with the project open next to the book.

The project

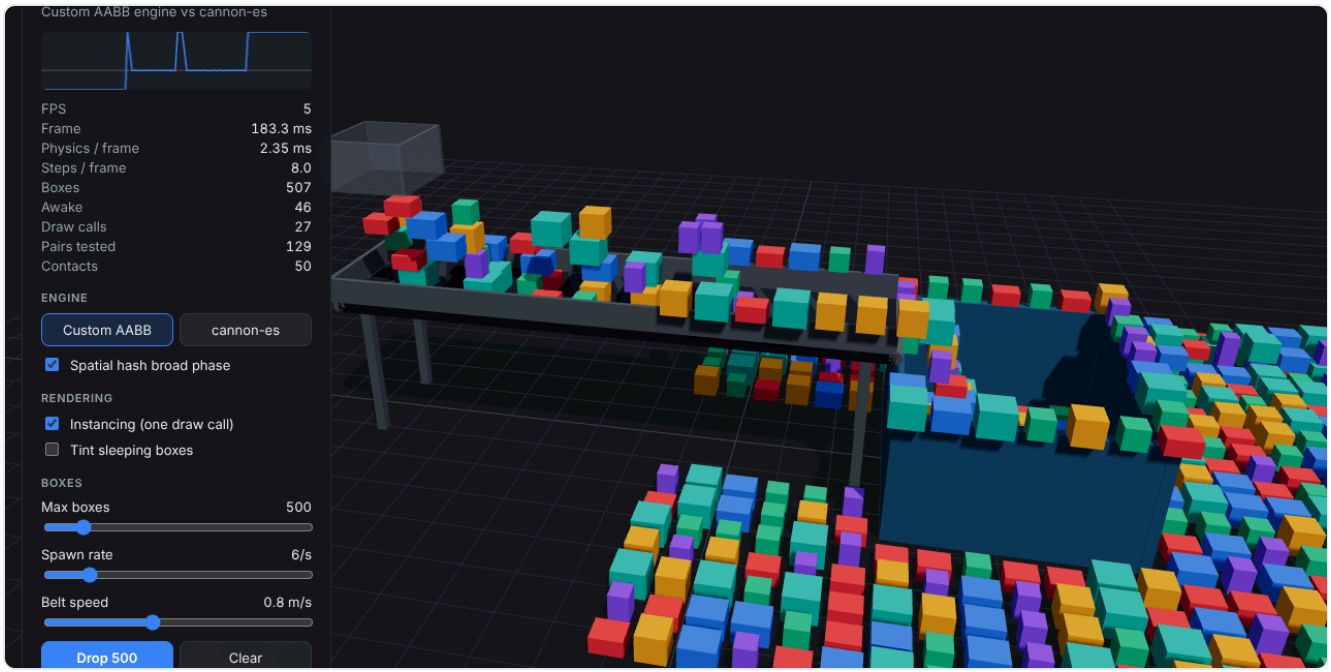
	Project	Stack	What it shows
3	**Conveyor + Physics**	Three.js · cannon-es · TypeScript	A physics engine written from scratch, the same scene on cannon-es, and the optimisations that let thousands of boxes run at 60 fps in a browser.

CHAPTER 3 · `vention/snack3-conveyor-physics`

Conveyor + Physics

A physics engine written from scratch, the same scene on cannon-es, and the optimisations that let thousands of boxes run at 60 fps in a browser.

Three.js · cannon-es · TypeScript



After a 500-box drop: boxes still ride the belt into the bin while the pile around it settles.

The one idea to take away

Physics runs at a fixed 120 Hz, decoupled from the frame rate, and rendering blends the last two states:

`frame time` → `FixedStepper.advance` → `N × step(1/120 s)` → `alpha` → `BoxRenderer: 1 draw call`

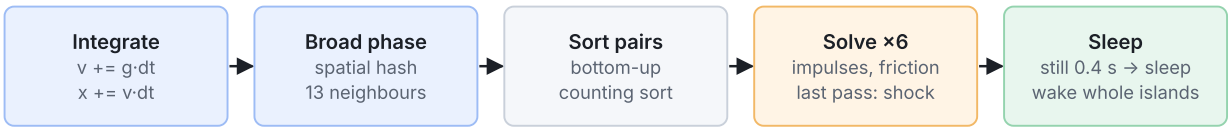
Each step is a handful of stages, each with one optimisation that keeps it cheap:

`integrate` → `broad phase (spatial hash)` → `sort bottom-up` → `6 solver passes, last with shock` → `sleep`

CONCEPTS · 3.1

What a physics engine does in one step

A rigid-body engine keeps each body's position and velocity (and, for bodies that rotate, orientation, angular velocity and an inertia tensor) and advances them by one small time step at a time. Every step has the same shape:



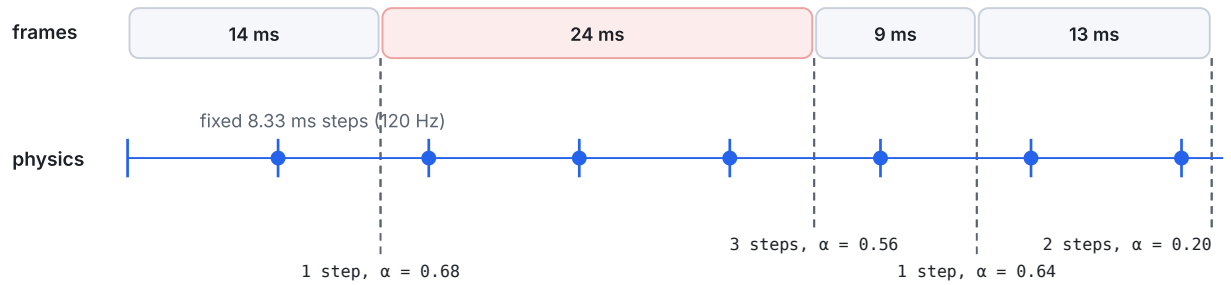
One fixed step of the custom engine (AabbWorld.step). Sleeping boxes skip integration and solving, so a settled pile costs almost nothing.

Figure 3.1. The four stages of a physics step, plus the sort that makes stacks stable.

The custom engine's big simplification is that boxes **never rotate**. Every box is an axis-aligned bounding box (AABB), so a contact normal is always one of the three axes and no contact produces torque. That cuts the code to a few hundred lines while keeping the parts that matter for a conveyor: stacking, friction, sleeping and scale.

CONCEPTS · 3.2

Time: fixed steps and interpolation



Each frame banks its time and spends it in whole steps; the leftover fraction α blends the last two physics states for drawing.

Figure 3.2. Uneven frames, identical physics steps. The 24 ms frame (a hitch) runs three steps to catch up.

- **Stability** depends on the step size. A solver tuned for 1/120 s may explode at 1/30 s. Fixed steps mean it only ever sees one size.
- **Determinism**: same inputs and same steps give the same results, so tests and benchmarks are reproducible.

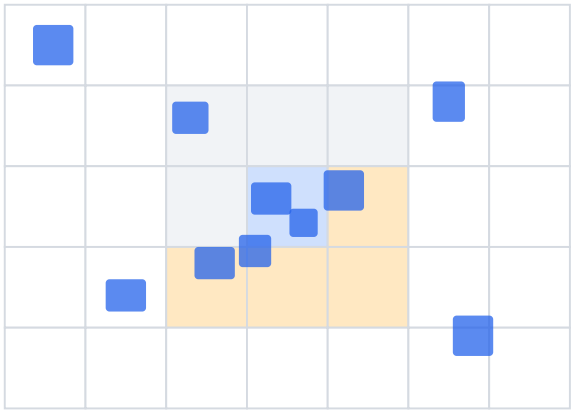
- **Interpolation** draws the moment between the last two physics states, using α. The cost is up to one step (8.3 ms) of extra latency; the alternative, extrapolating ahead, has no latency but visibly overshoots on collisions.

Integration is **semi-implicit Euler**: update velocity from forces first, then position from the new velocity. It costs the same as plain Euler but conserves energy far better, which is why almost every game engine uses it.

CONCEPTS · 3.3

Finding pairs: the broad phase

Testing every pair of n boxes is n(n − 1)/2 checks: 4.5 million for 3,000 boxes, every step. The **broad phase** cheaply finds the few pairs that might touch, and the **narrow phase** tests only those.



Cell size ≥ the largest box (+ margin), so two touching boxes are in the same or adjacent cells.

In 2D that's 8 neighbours; checking only the 4 "after" it finds each pair once. In 3D: 26 and 13.

Each bucket is a linked list in two typed arrays:



`next[7] = 3, next[3] = 12, next[12] = −1`

No list objects: rebuilding the grid each step is two array writes per box, and allocates nothing.

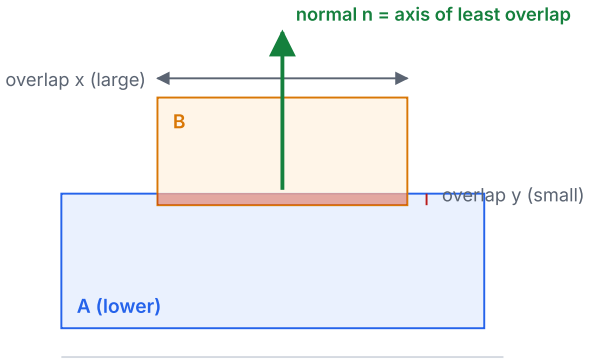
blue: the box's cell · orange: neighbours it checks · grey: neighbours that check it

Figure 3.3. The broad phase. A uniform grid stored in a hash table turns the n^2 all-pairs check into roughly n.

- **Uniform grid / spatial hash** (used here): O(n) when objects are similar in size. Struggles when sizes vary a lot.
- **Sweep and prune** (cannon-es): sort bounds along an axis and compare neighbours. Fast when objects move a little each step, because the sort is nearly sorted already.
- **Dynamic AABB tree** (Box2D, Bullet, PhysX): a hierarchy of bounding boxes. Handles any sizes and scales well; more complex to maintain.

CONCEPTS · 3.4

Contacts and impulses



`vn = (v_B − v_A) · n`
closing speed along the normal (< 0 = approaching)

`j = −(1 + e) · vn / (invA + invB)`
impulse that stops (or bounces) the approach

`|jt| ≤ μ · j`
Coulomb: friction can't exceed μ × the push

`v_rel = v_box − v_surface`
a belt's surface moves, so friction drags boxes

`push out 80% of (overlap − slop)`
fix overlap gently, without jitter

Figure 3.4. One contact between two axis-aligned boxes: find the normal, cancel the closing speed, cap friction, and remove the overlap.

Each contact is a constraint: “these two may not approach along the normal”. Solving all of them at once is a large linear problem, so engines use **sequential impulses** (projected Gauss–Seidel): fix each contact in turn, then sweep again, because fixing one disturbs its neighbours. More iterations give stiffer stacks at more cost; six is the trade-off here.

- **Restitution** only above 0.5 m/s, so resting contacts don't bounce forever on tiny gravity-driven speeds.
- **Position correction** pushes out most of the overlap beyond a small **slop**. Correcting to exactly zero makes resting boxes jitter.
- **Coulomb friction** caps the sideways impulse at μ times the normal one. Below the cap the slide stops (static friction); above it, the box keeps sliding.
- **Warm starting**, which real engines add, reuses last step's impulses as the starting guess, so fewer iterations converge. It's an exercise below.

CONCEPTS · 3.5

Stacks, sleeping and islands

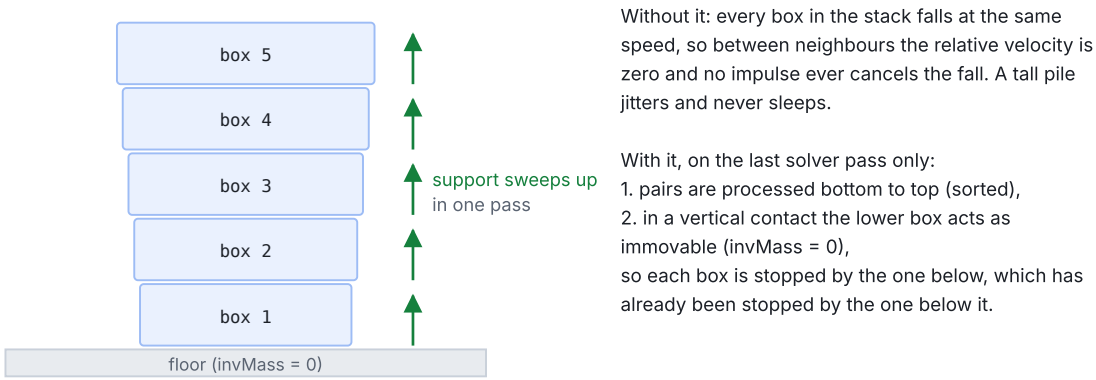


Figure 3.5. Shock propagation: the trick that lets a 3,000-box pile settle and sleep within about two seconds.

Sleeping is the other half of scaling: a box that has been nearly still for 0.4 s stops being simulated. The danger is waking: if something is removed from under a sleeping pile, the boxes above must wake, or they'd hang in the air. Waking therefore spreads to everything touching, then everything touching those, the whole connected **island**. Two bugs found while building this were both about waking: boxes falling asleep on a stopped belt and ignoring it when it restarted, and a wake stack that overwrote itself while spreading.

CONCEPTS · 3.6

Performance in a browser

- **Structure of arrays**: one typed array per property (`px`, `py`, `vx`…) instead of one object per box. Loops read contiguous memory, which suits CPU caches, and the garbage collector has nothing to scan.
- **No allocation while running**: pools, preallocated pair buffers, scratch objects. Allocation-free code has no GC pauses.
- **One draw call for all boxes**: `InstancedMesh` sends one geometry plus a buffer of per-box matrices and colors. With ~500 boxes visible, unticking instancing took the scene from 27 draw calls to 484.
- **Measure, don't guess**: time each phase with `performance.now()`, warm the JIT up before benchmarking, fix the random seed. 3,000 boxes cost about 1.6 ms per physics step while falling and 0.8 ms once settled.

CHAPTER 3

Line by line

12 parts, in the order data flows through the code.

PART 3.1 · `src/layout.ts`

The work cell as data

Every dimension in one file. The physics engines build their colliders from it and the renderer draws the same boxes, so what you see is exactly what collides.

Simulation constants

src/layout.ts · lines 6–16

```
export const GRAVITY = -9.81;
export const FIXED_STEP = 1 / 120; // physics runs at 120 Hz whatever the display does
export const MAX_SUBSTEPS = 8; // per frame; beyond this we drop time instead of spiralling
export const MAX_BODIES = 4096;
export const DENSITY = 500; // kg/m³, roughly cardboard boxes with something in them

export interface V3 {
  x: number;
  y: number;
  z: number;
}
```

- `GRAVITY` is in m/s². Negative because +Y is up.
- `FIXED_STEP` is 1/120 s: physics always advances in steps this size, whatever the display's frame rate.
- `MAX_SUBSTEPS` caps how many steps one frame may run before it gives up on catching up (see FixedStepper).
- `MAX_BODIES` sizes every per-box array once, up front. Nothing grows while the app runs.
- `DENSITY` turns a box's volume into its mass, so big boxes are heavier than small ones.
- `V3` is a plain {x, y, z}. Three.js's `Vector3` fits it, but so does any object, which keeps the physics free of Three.js.

Static colliders

src/layout.ts · lines 18–26

```
export interface StaticCollider {
  name: string;
  min: V3;
  max: V3;
  /** The conveyor belt: its surface drags boxes along at the belt speed through friction. */
  conveyor?: boolean;
  /** Draw it? The floor is drawn separately as a grid. */
  visible?: boolean;
}
```

A static collider is an axis-aligned box given by its `min` and `max` corners, and it never moves. The `?` marks optional fields:

- `conveyor` marks the belt, whose surface drags boxes along.
- `visible: false` hides the floor, since the grid is drawn instead.

Belt, bin and spawner

src/layout.ts · lines 28–30

```
export const BELT = { minX: -1.6, maxX: 0.6, halfWidth: 0.25, topY: 0.6, thickness: 0.05 };
export const BIN = { minX: 0.6, maxX: 1.5, halfDepth: 0.55, height: 0.5, wall: 0.04 };
export const SPAWN = { x: -1.45, y: 0.95, zJitter: 0.14 };
```

The belt runs along X from −1.6 m to 0.6 m, 50 cm wide, with its top 60 cm above the floor. The bin starts exactly where the belt ends, so boxes falling off the end land inside it. Boxes appear at `SPAWN`, slightly above the belt's start, with a little random sideways offset (`zJitter`).

The floor and the belt

src/layout.ts · lines 35–43

```
export const STATICS: readonly StaticCollider[] = [
  // A thick floor, so fast-falling boxes can't step right through it in one physics step.
  { name: "floor", min: { x: -8, y: -1, z: -8 }, max: { x: 8, y: 0, z: 8 }, visible: false },
  {
    name: "belt",
    min: { x: BELT.minX, y: BELT.topY - BELT.thickness, z: -BELT.halfWidth },
    max: { x: BELT.maxX, y: BELT.topY, z: BELT.halfWidth },
    conveyor: true,
  },
```

The floor is a whole meter thick even though only its top matters. A box falling at 8 m/s moves about 7 cm per physics step, so a thin floor could be stepped over completely between two checks (“tunnelling”). A thick one can't.

The belt is just a thin slab whose top is at `BELT.topY`, flagged `conveyor: true`. The rails, the backstop and the four bin walls that follow are built the same way.

Box sizes

src/layout.ts · lines 82–94

```
/**
 * A handful of box sizes (half extents). Using presets rather than random sizes
 * lets the cannon-es backend share one shape per size, and gives each size a color.
 */
export const SIZE_PRESETS: readonly V3[] = [
  { x: 0.05, y: 0.05, z: 0.05 },
  { x: 0.06, y: 0.04, z: 0.05 },
  { x: 0.04, y: 0.04, z: 0.04 },
  { x: 0.055, y: 0.03, z: 0.045 },
  { x: 0.035, y: 0.05, z: 0.035 },
  { x: 0.06, y: 0.05, z: 0.06 },
];
export const MAX_HALF_EXTENT = 0.06;
```

Six fixed sizes (half extents, so the first is a 10 cm cube). Using presets rather than any random size means cannon-es can share one shape object per size, and each size gets its own color. `MAX_HALF_EXTENT` is the biggest half size; the spatial hash uses it to choose its cell size.

Stress-test area and world bounds

src/layout.ts · lines 96–100

```
/** Where a stress-test burst rains down: over the belt end, the bin and the floor around it.
*/
export const BURST_AREA = { minX: -0.2, maxX: 2.6, minZ: -1.4, maxZ: 1.4, baseY: 1.0, spacing:
0.14 };

/** Boxes outside this are removed (fell off the world, or got flung away). */
export const BOUNDS = { minY: -2, maxAbsXZ: 6 };
```

A burst lays boxes out in a grid of layers starting 1 m up, 14 cm apart. The largest box is 12 cm, so none of them start overlapping. Anything that falls below −2 m or drifts more than 6 m out is removed.

PART 3.2 · `src/physics/FixedStepper.ts`

The fixed-timestep loop

Frames arrive at uneven intervals. Physics shouldn't care: this turns whatever time a frame took into a whole number of identical 1/120 s steps.

State

src/physics/FixedStepper.ts · lines 10–22

```
export class FixedStepper {
  readonly step: number;
  readonly maxSteps: number;
  accumulator = 0;
  /** How far between the previous and current physics state this frame is (0..1). */
  alpha = 0;
  /** Simulated time thrown away because a frame needed more than maxSteps. */
  droppedTime = 0;

  constructor(step: number, maxSteps: number) {
    this.step = step;
    this.maxSteps = maxSteps;
  }
```

`accumulator` holds simulated time that has built up but not been stepped yet. `alpha` is the fraction of a step left over after the last one, from 0 to 1, which the renderer uses to blend. `droppedTime` counts time thrown away because a frame was too slow.

Running the steps

src/physics/FixedStepper.ts · lines 24–32

```
/** Runs as many fixed steps as `frameDt` calls for. Returns how many ran. */
advance(frameDt: number, stepOnce: () => void): number {
  this.accumulator += frameDt;
  let steps = 0;
  while (this.accumulator >= this.step && steps < this.maxSteps) {
    stepOnce();
    this.accumulator -= this.step;
    steps++;
  }
```

Add the frame's time to the bank, then spend it a step at a time. On a 60 Hz display each frame adds 1/60 s and two 1/120 s steps run; on a 144 Hz display most frames run zero or one. Either way every step is the same size, so the simulation behaves identically. `stepOnce` is a callback, so this class knows nothing about the engine it drives.

Avoiding the spiral of death

src/physics/FixedStepper.ts · lines 33–38

```
// If physics can't keep up, drop the backlog rather than trying to catch up
// next frame, which would take even longer (the "spiral of death").
if (this.accumulator >= this.step) {
  this.droppedTime += this.accumulator - (this.accumulator % this.step);
  this.accumulator %= this.step;
}
```

If a frame was very slow, the bank holds more than `maxSteps` worth. Trying to run it all next frame would make that frame slower still, which leaves even more to run the frame after: the “spiral of death”. Instead, whole steps beyond the cap are dropped (and counted), keeping only the fraction of a step. `%` is the remainder after division.

Interpolation factor

src/physics/FixedStepper.ts · lines 39–41

```
    this.alpha = this.accumulator / this.step;
    return steps;
  }
```

The leftover fraction of a step. If `alpha` is 0.3, the moment being drawn is 30% of the way from the previous physics state to the current one, so each box is drawn 30% of the way between its two positions. Without this, motion would visibly stutter whenever the frame rate and the physics rate don't line up.

GOING DEEPER

How far does determinism go?

Fixed steps plus a seeded random generator make a run repeatable on one machine and one build. Across browsers or CPUs, results can still drift: `Math.sin` and friends aren't required to round identically everywhere, and different JIT decisions can reorder floating-point operations. Networked games that simulate in lockstep therefore use fixed-point math or carefully specified math libraries; for a single-player demo and its tests, same-machine determinism is enough.

PART 3.3 · `src/sim/SlotPool.ts`

Object pooling

Boxes are created and removed constantly. Instead of allocating objects, every box is a slot number into preallocated arrays.

Three arrays

src/sim/SlotPool.ts · lines 13–31

```
export class SlotPool {
  readonly capacity: number;
  /** Live slots, packed: dense[0..count-1]. */
  readonly dense: Int32Array;
  count = 0;
  /** slot -> its index in `dense`, or -1 if free. */
  private readonly where: Int32Array;
  private readonly free: Int32Array;
  private freeTop: number;

  constructor(capacity: number) {
    this.capacity = capacity;
    this.dense = new Int32Array(capacity);
    this.where = new Int32Array(capacity).fill(-1);
    this.free = new Int32Array(capacity);
    // Stack the free slots so slot 0 comes out first.
    for (let i = 0; i < capacity; i++) this.free[i] = capacity - 1 - i;
    this.freeTop = capacity;
  }
```

- `dense` lists the live slots packed at the front, so loops run over exactly `count` entries and never skip gaps.
- `where` maps a slot back to its position in `dense` (−1 when free). That's what makes removal O(1).
- `free` is a stack of unused slots. The constructor fills it backwards so slot 0 is handed out first.

`Int32Array` is a fixed-size array of 32-bit integers: compact, fast, and never resized or garbage-collected.

Taking a slot

src/sim/SlotPool.ts · lines 33–40

```
/** Returns a free slot, or -1 if the pool is full. */
alloc(): number {
  if (this.freeTop === 0) return -1;
  const slot = this.free[--this.freeTop];
  this.where[slot] = this.count;
  this.dense[this.count++] = slot;
  return slot;
}
```

Pop a slot off the free stack, append it to the end of `dense`, and record where it went. `--this.freeTop` decreases the counter first and then reads; `this.count++` reads the old value and then increases it.

Giving it back: swap-remove

src/sim/SlotPool.ts · lines 42–50

```
release(slot: number): void {
  const k = this.where[slot];
  if (k < 0) return; // already free
  const last = this.dense[--this.count];
  this.dense[k] = last; // move the last live slot into the hole
  this.where[last] = k;
  this.where[slot] = -1;
  this.free[this.freeTop++] = slot;
}
```

To remove from the middle of `dense` without shifting everything after it, move the **last** live slot into the hole and shrink the count. One copy instead of up to 4,000. The cost is that `dense` isn't kept in spawn order, which nothing relies on. The slot goes back on the free stack for reuse.

PART 3.4 · `src/physics/types.ts`

One interface, two engines

The rest of the app only ever talks to this interface, which is what makes the engines swappable and the benchmark fair.

Data types

src/physics/types.ts · lines 11–28

```
/** A box to add: center and half extents, in meters. */
export interface BodyDesc {
  x: number;
  y: number;
  z: number;
  hx: number;
  hy: number;
  hz: number;
}

export interface PhysicsStats {
  /** Pairs checked in the narrow phase this step (custom engine only; -1 when not available).
*/
  pairsTested: number;
  /** Touching pairs this step. */
  contacts: number;
  /** Bodies not asleep. */
  awake: number;
}
```

`BodyDesc` describes a new box: its center and half extents. `PhysicsStats` is what the panel shows. cannon-es doesn't report how many pairs it tested, so it reports −1, and the panel shows “n/a”.

PhysicsBackend

src/physics/types.ts · lines 30–48

```
/**
 * What the rest of the app needs from a physics engine. The custom engine and
 * cannon-es both implement it, so they can be swapped at runtime and benchmarked
 * on exactly the same scene.
 */
export interface PhysicsBackend {
  readonly name: string;
  readonly stats: PhysicsStats;
  beltSpeed: number;
  add(slot: number, desc: BodyDesc): void;
  remove(slot: number): void;
  /** Remember the current state as "previous", for interpolation. Call before each step. */
  savePrevious(pool: SlotPool): void;
  step(dt: number, pool: SlotPool): void;
  getPosition(slot: number, out: V3): void;
  /** Position and rotation blended `alpha` of the way from the previous step to the current
one. */
  readTransform(slot: number, alpha: number, pos: V3, quat: Quat): void;
  isAsleep(slot: number): boolean;
}
```

Everything an engine must do, keyed by slot number: add and remove boxes, step, report positions, and say whether a box is asleep. `savePrevious` plus `readTransform(slot, alpha, …)` is the interpolation contract: the engine keeps the last two states and blends them for the renderer.

`beltSpeed` is a property, so the custom engine can implement it with a getter and setter (it wakes everything on change).

PART 3.5 · `src/physics/SpatialHash.ts`

The spatial hash

The broad phase answers “which pairs of boxes might be touching?”. Checking every pair is n^2 work; a grid makes it roughly n.

Layout in memory

src/physics/SpatialHash.ts · lines 12–33

```
export class SpatialHash {
  readonly cellSize: number;
  readonly head: Int32Array;
  readonly next: Int32Array;
  readonly cx: Int32Array;
  readonly cy: Int32Array;
  readonly cz: Int32Array;
  private readonly inv: number;
  private readonly mask: number;

  constructor(capacity: number, cellSize: number) {
    this.cellSize = cellSize;
    this.inv = 1 / cellSize;
    let size = 1;
    while (size < capacity * 2) size <<= 1; // power of two, so `& mask` replaces `%`
    this.mask = size - 1;
    this.head = new Int32Array(size);
    this.next = new Int32Array(capacity);
    this.cx = new Int32Array(capacity);
    this.cy = new Int32Array(capacity);
    this.cz = new Int32Array(capacity);
  }
```

Space is divided into cubic cells, and each box belongs to the cell containing its center. Since there could be any number of cells, they're stored in a hash table with a fixed number of buckets (at least twice the number of boxes, so buckets are rarely shared).

Each bucket is a linked list built from two arrays: `head[bucket]` is the first box in it and `next[box]` is the box after that, with −1 meaning the end. No list objects, no allocation. `cx`, `cy`, `cz` remember each box's cell.

Rounding the table size up to a power of two (`size <<= 1` doubles it) means `& mask` can replace the slower `%` when turning a hash into a bucket number.

Hashing a cell

src/physics/SpatialHash.ts · lines 35–38

```
/** Bucket for integer cell coordinates. Large primes spread neighboring cells apart. */
bucket(x: number, y: number, z: number): number {
  return (Math.imul(x, 73856093) ^ Math.imul(y, 19349663) ^ Math.imul(z, 83492791)) &
this.mask;
}
```

Multiplies each cell coordinate by a large prime, mixes the three with XOR (`^`), and keeps the low bits. Neighboring cells land in scattered buckets. `Math.imul` does 32-bit integer multiplication, wrapping around as C would, instead of producing a huge floating-point number.

Filling the grid

src/physics/SpatialHash.ts · lines 40–56

```
/** Integer cell coordinate along one axis. */
cell(v: number): number {
  return Math.floor(v * this.inv);
}

build(list: Int32Array, count: number, px: Float64Array, py: Float64Array, pz: Float64Array):
void {
  this.head.fill(-1);
  for (let k = 0; k < count; k++) {
    const i = list[k];
    const x = (this.cx[i] = this.cell(px[i]));
    const y = (this.cy[i] = this.cell(py[i]));
    const z = (this.cz[i] = this.cell(pz[i]));
    const b = this.bucket(x, y, z);
    this.next[i] = this.head[b];
    this.head[b] = i;
  }
}
```

`cell` turns a coordinate into an integer cell index (multiplying by `1/cellSize` is cheaper than dividing). `build` empties every bucket, then for each live box records its cell and pushes it onto the front of its bucket's list. This runs once per physics step.

Half the neighbors

src/physics/SpatialHash.ts · lines 59–72

```
/**
 * Half of the 26 neighbor offsets: the ones "after" (0,0,0) in x, then y, then z
 * order. Checking only these (plus the body's own cell) visits every neighboring
 * pair exactly once.
 */
export const HALF_NEIGHBORS: readonly (readonly [number, number, number])[] = (() => {
  const out: [number, number, number][] = [];
  for (let dx = -1; dx <= 1; dx++)
    for (let dy = -1; dy <= 1; dy++)
      for (let dz = -1; dz <= 1; dz++) {
        if (dx > 0 || (dx === 0 && dy > 0) || (dx === 0 && dy === 0 && dz > 0)) out.push([dx,
dy, dz]);
      }
  return out;
})();
```

Cells are at least as wide as the largest box, so two touching boxes are always in the same or adjacent cells: 27 cells to look at. But looking in all 26 neighbors from every box finds each pair twice (A finds B, then B finds A).

So each box only looks at the 13 neighbors that come “after” it in x, then y, then z order, plus later boxes in its own cell. Every adjacent pair is then found exactly once. The `(() => { … })()` runs once at startup to build the list.

GOING DEEPER

Choosing the cell size

Too small, and a box overlaps many cells, so it must be inserted into all of them or neighbours missed. Too large, and many boxes share a cell, sliding back toward n^2. Here the cell is the largest box plus the broad-phase margin, and each box is inserted by its center only, which is exactly enough to guarantee touching boxes are in neighbouring cells.

The three multipliers in `bucket()` (73856093, 19349663, 83492791) are the classic primes from Teschner et al.'s 2003 paper on spatial hashing for collision detection. Different cells can still collide in one bucket, which the code handles by checking each box's stored cell coordinates.

PART 3.6 · `src/physics/AabbWorld.ts`

The custom engine

About 500 lines that do what a physics engine does, for boxes that never rotate. The four stages of a step are integration, broad phase, narrow phase and sleeping.

Tuning constants

src/physics/AabbWorld.ts · lines 6–21

```
// ---- Tuning ----
const ITERATIONS = 6; // solver passes per step; more = stiffer stacks, more cost
const RESTITUTION = 0.1; // bounciness for real impacts
const BOUNCE_THRESHOLD = 0.5; // m/s; slower contacts don't bounce, which stops resting jitter
const FRICTION = 0.5;
const SLOP = 0.0005; // allowed overlap (m) before position correction kicks in
const CORRECTION = 0.8; // fraction of the overlap removed per pass
const BROAD_MARGIN = 0.005; // m; pairs this close become candidates for the step
const MAX_SPEED = 8; // m/s; caps how far a box can move in one step (anti-tunnelling)
const SLEEP_SPEED_SQ = 0.06 * 0.06;
const SLEEP_TIME = 0.4; // s below SLEEP_SPEED before a body sleeps
const WAKE_SPEED = 0.25; // m/s impact that wakes a sleeping body
const WAKE_MARGIN = 0.01; // m; sleeping bodies this close to a woken one wake too
const MAX_PAIRS_PER_BODY = 16;
const LAYER_HEIGHT = 0.05; // m; pairs are ordered bottom-up in layers this thick
const LAYERS = 128; // covers y from -1 m to 5.4 m; anything higher goes in the top layer
```

Each has a comment saying what it trades off. A few worth knowing:

- `ITERATIONS`: contacts are solved 6 times per step. Each pass fixes some overlap and disturbs neighbors a little; repeating converges.
- `SLOP`: up to 0.5 mm of overlap is allowed before correction. Correcting to exactly zero makes resting boxes jitter.
- `BOUNCE_THRESHOLD`: only impacts faster than 0.5 m/s bounce. Otherwise resting boxes would keep hopping on tiny gravity-driven contacts.
- `MAX_SPEED`: capping speed caps how far a box moves per step, the other half of the anti-tunnelling defence.

Structure of arrays

src/physics/AabbWorld.ts · lines 47–82

```
export class AabbWorld implements PhysicsBackend {
  readonly name = "Custom AABB";
  readonly stats: PhysicsStats = { pairsTested: 0, contacts: 0, awake: 0 };
  /** Off = test every pair of bodies (O(n²)), to show what the spatial hash saves. */
  useSpatialHash = true;

  readonly px: Float64Array;
  readonly py: Float64Array;
  readonly pz: Float64Array;
  readonly vx: Float64Array;
  readonly vy: Float64Array;
  readonly vz: Float64Array;
  readonly hx: Float64Array;
  readonly hy: Float64Array;
  readonly hz: Float64Array;
  readonly invMass: Float64Array;
  readonly asleep: Uint8Array;
  private readonly alive: Uint8Array;
  private readonly prevX: Float64Array;
  private readonly prevY: Float64Array;
  private readonly prevZ: Float64Array;
  private readonly sleepTime: Float64Array;
  /** Set during a step for bodies touching the belt: those must not fall asleep while it runs.
*/
  private readonly onBelt: Uint8Array;
  private speed = 0.8;

  private readonly statics: StaticBox[];
  private readonly hash: SpatialHash;
  private readonly neighborOffsets: Int32Array;
  private pairs: Int32Array;
  private sortedPairs: Int32Array;
  private readonly layerCounts = new Int32Array(LAYERS + 1);
  private pairCount = 0;
  private readonly wakeStack: Int32Array;
  private hashBuilt = false;
  private lastPool: SlotPool | null = null;
```

There's no `Box` class. Each property of every box is its own typed array indexed by slot: `px[7]` is box 7's x position. Loops that touch one property read contiguous memory, which is much faster for the CPU's cache than chasing objects scattered around the heap, and there is nothing for the garbage collector to scan. This is how engines and ECS frameworks lay out data.

`invMass` stores 1/mass, because the solver divides by mass constantly, and an immovable object is simply `invMass = 0`.

Constructor

src/physics/AabbWorld.ts · lines 84–112

```
constructor(capacity: number, statics: readonly StaticCollider[] = STATICS) {
  const f64 = () => new Float64Array(capacity);
  this.px = f64(); this.py = f64(); this.pz = f64();
  this.vx = f64(); this.vy = f64(); this.vz = f64();
  this.hx = f64(); this.hy = f64(); this.hz = f64();
  this.prevX = f64(); this.prevY = f64(); this.prevZ = f64();
  this.invMass = f64();
  this.sleepTime = f64();
  this.asleep = new Uint8Array(capacity);
  this.alive = new Uint8Array(capacity);
  this.onBelt = new Uint8Array(capacity);

  this.statics = statics.map((s) => ({
    cx: (s.min.x + s.max.x) / 2,
    cy: (s.min.y + s.max.y) / 2,
    cz: (s.min.z + s.max.z) / 2,
    hx: (s.max.x - s.min.x) / 2,
    hy: (s.max.y - s.min.y) / 2,
    hz: (s.max.z - s.min.z) / 2,
    conveyor: s.conveyor === true,
  }));

  // Cells as wide as the biggest box (plus margin), so touching boxes are always in
neighboring cells.
  this.hash = new SpatialHash(capacity, 2 * MAX_HALF_EXTENT + BROAD_MARGIN);
  this.neighborOffsets = Int32Array.from(HALF_NEIGHBORS.flat());
  this.pairs = new Int32Array(capacity * MAX_PAIRS_PER_BODY * 2);
  this.sortedPairs = new Int32Array(capacity * MAX_PAIRS_PER_BODY * 2);
  this.wakeStack = new Int32Array(capacity);
}
```

Allocates everything once at full capacity: positions, velocities, sizes, previous positions for interpolation, sleep timers and flags. Static colliders are converted from min/max corners to center + half size, which is what the collision math wants.

The hash's cell size is the largest box plus the broad-phase margin, which is what guarantees touching boxes are in neighboring cells. The pair buffer holds up to 16 pairs per box, again allocated once.

Belt speed

src/physics/AabbWorld.ts · lines 114–129

```
get beltSpeed(): number {
  return this.speed;
}

/** Changing the belt speed wakes everything: sleeping boxes on the belt have to start (or
stop) moving. */
set beltSpeed(v: number) {
  if (v === this.speed) return;
  this.speed = v;
  if (this.lastPool) {
    const { dense, count } = this.lastPool;
    for (let k = 0; k < count; k++) {
      this.asleep[dense[k]] = 0;
      this.sleepTime[dense[k]] = 0;
    }
  }
}
```

A getter/setter pair, so `world.beltSpeed = 0.5` runs code. Changing the speed wakes every box: a sleeping box on the belt isn't being simulated, so without this it would ignore the belt starting up again.

Adding and removing

src/physics/AabbWorld.ts · lines 131–149

```
add(slot: number, d: BodyDesc): void {
  this.px[slot] = this.prevX[slot] = d.x;
  this.py[slot] = this.prevY[slot] = d.y;
  this.pz[slot] = this.prevZ[slot] = d.z;
  this.vx[slot] = this.vy[slot] = this.vz[slot] = 0;
  this.hx[slot] = d.hx;
  this.hy[slot] = d.hy;
  this.hz[slot] = d.hz;
  this.invMass[slot] = 1 / (8 * d.hx * d.hy * d.hz * DENSITY);
  this.asleep[slot] = 0;
  this.sleepTime[slot] = 0;
  this.alive[slot] = 1;
}

remove(slot: number): void {
  this.alive[slot] = 0;
  // Whatever was resting on this box has lost its support: wake it (and its pile).
  this.drainWake(this.pushSleepingNeighbors(slot, 0));
}
```

`add` writes the new box's values into its slot. Previous position = current position, so interpolation doesn't draw a streak from wherever the slot's last box was. Mass is volume × density; `8·hx·hy·hz` is the volume, since the full sides are twice the half extents.

`remove` marks the slot dead, then wakes anything that was resting on it (explained under Sleeping).

One step

src/physics/AabbWorld.ts · lines 151–180

```
savePrevious(pool: SlotPool): void {
  const { dense, count } = pool;
  for (let k = 0; k < count; k++) {
    const i = dense[k];
    this.prevX[i] = this.px[i];
    this.prevY[i] = this.py[i];
    this.prevZ[i] = this.pz[i];
  }
}
```

```
step(dt: number, pool: SlotPool): void {
  this.lastPool = pool;
  this.integrate(dt, pool);
  this.broadPhase(pool);
  this.sortPairsBottomUp();

  this.stats.contacts = 0;
  for (let k = 0; k < pool.count; k++) this.onBelt[pool.dense[k]] = 0;
  for (let it = 0; it < ITERATIONS; it++) {
    const first = it === 0;
    const shock = it === ITERATIONS - 1;
    for (let k = 0; k < pool.count; k++) {
      const i = pool.dense[k];
      if (!this.asleep[i]) this.solveStatics(i, first);
    }
    for (let p = 0; p < this.pairCount; p++) this.solvePair(this.pairs[2 * p], this.pairs[2 * p
+ 1], first, shock);
  }

  this.updateSleep(dt, pool);
}
```

`savePrevious` copies positions aside before the step, for interpolation. `step` is the whole engine in a dozen lines:

1. **Integrate**: move everything by gravity and velocity.
2. **Broad phase**: find the pairs that might touch.
3. **Sort** those pairs from bottom to top.
4. **Solve**, 6 times: each awake box against the statics, then every pair. The first pass counts contacts; the last uses shock propagation.
5. **Sleep**: put still boxes to sleep.

`onBelt` is cleared first and set again during solving, so it's true only for boxes touching the belt this step.

Reading results

src/physics/AabbWorld.ts · lines 182–198

```
getPosition(slot: number, out: V3): void {
  out.x = this.px[slot];
  out.y = this.py[slot];
  out.z = this.pz[slot];
}

readTransform(slot: number, alpha: number, pos: V3, quat: Quat): void {
  pos.x = this.prevX[slot] + (this.px[slot] - this.prevX[slot]) * alpha;
  pos.y = this.prevY[slot] + (this.py[slot] - this.prevY[slot]) * alpha;
  pos.z = this.prevZ[slot] + (this.pz[slot] - this.prevZ[slot]) * alpha;
  quat.x = quat.y = quat.z = 0; // axis-aligned: never rotates
  quat.w = 1;
}

isAsleep(slot: number): boolean {
  return this.asleep[slot] === 1;
}
```

`readTransform` blends previous and current positions by `alpha` (the same lerp formula as Snack 2). Rotation is always the identity quaternion (0, 0, 0, 1), because these boxes never rotate.

1. Integration

src/physics/AabbWorld.ts · lines 208–228

```
// ---- 1. Integration ----

private integrate(dt: number, pool: SlotPool): void {
  const { px, py, pz, vx, vy, vz, asleep } = this;
  const { dense, count } = pool;
  for (let k = 0; k < count; k++) {
    const i = dense[k];
    if (asleep[i]) continue;
    vy[i] += GRAVITY * dt;
    const speedSq = vx[i] * vx[i] + vy[i] * vy[i] + vz[i] * vz[i];
    if (speedSq > MAX_SPEED * MAX_SPEED) {
      const s = MAX_SPEED / Math.sqrt(speedSq);
      vx[i] *= s;
      vy[i] *= s;
      vz[i] *= s;
    }
    px[i] += vx[i] * dt;
    py[i] += vy[i] * dt;
    pz[i] += vz[i] * dt;
  }
}
```

Semi-implicit Euler: update velocity first (gravity), then move by the **new** velocity. Doing it in that order, rather than moving by the old velocity, is what keeps simple game physics stable. Sleeping boxes are skipped entirely.

Copying the arrays into local constants (`const { px, … } = this`) saves a property lookup on every access in the loop. Speeds above `MAX_SPEED` are scaled back down.

2. Broad phase

src/physics/AabbWorld.ts · lines 230–267

```
// ---- 2. Broad phase ----

/** Fills `pairs` with every pair of live bodies whose boxes are within BROAD_MARGIN. */
private broadPhase(pool: SlotPool): void {
  this.pairCount = 0;
  this.stats.pairsTested = 0;
  const { dense, count } = pool;
```

```
  if (!this.useSpatialHash) {
    for (let a = 0; a < count; a++)
      for (let b = a + 1; b < count; b++) this.candidate(dense[a], dense[b]);
    this.hashBuilt = false;
    return;
  }

  const h = this.hash;
  h.build(dense, count, this.px, this.py, this.pz);
  this.hashBuilt = true;
  const { head, next, cx, cy, cz } = h;
  const offsets = this.neighborOffsets;

  for (let k = 0; k < count; k++) {
    const i = dense[k];
    const x = cx[i], y = cy[i], z = cz[i];
    // Same cell: only bodies after i in the chain, so each pair is seen once.
    for (let j = next[i]; j !== -1; j = next[j]) {
      if (cx[j] === x && cy[j] === y && cz[j] === z) this.candidate(i, j);
    }
    // The 13 "forward" neighbor cells.
    for (let o = 0; o < offsets.length; o += 3) {
      const qx = x + offsets[o], qy = y + offsets[o + 1], qz = z + offsets[o + 2];
      for (let j = head[h.bucket(qx, qy, qz)]; j !== -1; j = next[j]) {
        // Different cells can share a bucket; check it really is the cell we asked for.
        if (cx[j] === qx && cy[j] === qy && cz[j] === qz) this.candidate(i, j);
      }
    }
  }
}
```

With the hash turned off, the double loop tries every pair: for 3,000 boxes that's 4.5 million checks per step. With it, the grid is rebuilt and each box checks its own cell's later boxes and its 13 forward neighbors.

Different cells can hash to the same bucket, so each box found is checked against the exact cell asked for. Without that check, a bucket shared by two of a box's neighbor cells would be scanned twice and report pairs twice.

Sorting pairs bottom-up

src/physics/AabbWorld.ts · lines 269–292

```
/**
 * Reorders the pairs from lowest to highest (counting sort on height layers, O(n)).
 * Solving bottom-up lets support travel up a stack within a single pass.
 */
private sortPairsBottomUp(): void {
  const counts = this.layerCounts;
  counts.fill(0);
  const { py, pairs } = this;
  const layerOf = (p: number) => {
    const y = Math.min(py[pairs[2 * p]], py[pairs[2 * p + 1]]);
    return Math.min(LAYERS - 1, Math.max(0, Math.floor((y + 1) / LAYER_HEIGHT)));
  };
  for (let p = 0; p < this.pairCount; p++) counts[layerOf(p) + 1]++;
  for (let l = 1; l <= LAYERS; l++) counts[l] += counts[l - 1]; // running totals = where each
layer starts
  const out = this.sortedPairs;
  for (let p = 0; p < this.pairCount; p++) {
    const at = counts[layerOf(p)]++;
    out[2 * at] = pairs[2 * p];
    out[2 * at + 1] = pairs[2 * p + 1];
  }
  // Swap buffers instead of copying back.
  this.sortedPairs = pairs;
  this.pairs = out;
}
```

A **counting sort**: instead of comparing pairs, it buckets them by height in 5 cm layers. It counts how many pairs fall in each layer, turns the counts into running totals (where each layer starts), then drops each pair into its place. That's O(n), faster than a general sort's O(n log n).

Rather than copying the sorted pairs back, the two buffers swap roles. Why sort at all is explained under shock propagation below.

Candidate test

src/physics/AabbWorld.ts · lines 294–307

```
private candidate(i: number, j: number): void {
  if (this.asleep[i] && this.asleep[j]) return; // two sleepers can't affect each other
  this.stats.pairsTested++;
  if (
    Math.abs(this.px[j] - this.px[i]) < this.hx[i] + this.hx[j] + BROAD_MARGIN &&
    Math.abs(this.py[j] - this.py[i]) < this.hy[i] + this.hy[j] + BROAD_MARGIN &&
    Math.abs(this.pz[j] - this.pz[i]) < this.hz[i] + this.hz[j] + BROAD_MARGIN &&
    2 * this.pairCount < this.pairs.length
  ) {
    this.pairs[2 * this.pairCount] = i;
    this.pairs[2 * this.pairCount + 1] = j;
    this.pairCount++;
  }
}
```

Two sleeping boxes can't push each other, so the pair is skipped before it's even counted. Otherwise, two boxes are a candidate if they overlap (or nearly, within `BROAD_MARGIN`) on all three axes: the gap between centers is less than the sum of their half sizes. This one test is what an AABB (axis-aligned bounding box) check is.

3. Narrow phase: is there a contact, and which way?

src/physics/AabbWorld.ts · lines 311–334

```
/**
 * Box vs box. The normal points from i to j along the axis where they overlap least.
 *
 * `shock` (the last pass) is shock propagation: in a vertical contact the lower box
 * is treated as immovable, so a stack's upper boxes can't push it back down. Combined
 * with bottom-up order, the floor's support reaches the top of a stack in one sweep.
 * Without it, every box in a tall stack keeps a shared downward velocity that never
 * cancels, and the stack never comes to rest.
 */
private solvePair(i: number, j: number, first: boolean, shock: boolean): void {
  const { px, py, pz, vx, vy, vz, hx, hy, hz } = this;
  const dx = px[j] - px[i], dy = py[j] - py[i], dz = pz[j] - pz[i];
  const ox = hx[i] + hx[j] - Math.abs(dx);
  if (ox <= 0) return;
  const oy = hy[i] + hy[j] - Math.abs(dy);
  if (oy <= 0) return;
  const oz = hz[i] + hz[j] - Math.abs(dz);
  if (oz <= 0) return;
  if (first) this.stats.contacts++;

  let nx = 0, ny = 0, nz = 0, depth: number;
  if (ox <= oy && ox <= oz) { depth = ox; nx = dx < 0 ? -1 : 1; }
  else if (oy <= oz) { depth = oy; ny = dy < 0 ? -1 : 1; }
  else { depth = oz; nz = dz < 0 ? -1 : 1; }
```

Overlap on each axis is the sum of half sizes minus the distance between centers; if any is ≤ 0, they aren't touching. The contact **normal** is the axis with the **least** overlap: the direction that separates them with the smallest push. A box resting on another overlaps a little vertically and a lot horizontally, so the normal is vertical, as it should be.

Waking, and shock propagation

src/physics/AabbWorld.ts · lines 336–349

```
const rvx = vx[j] - vx[i], rvy = vy[j] - vy[i], rvz = vz[j] - vz[i];
const vn = rvx * nx + rvy * ny + rvz * nz; // < 0: closing in

// A sleeping box acts as immovable, unless it's hit hard enough to wake.
if (this.asleep[i] && vn < -WAKE_SPEED) this.wake(i);
if (this.asleep[j] && vn < -WAKE_SPEED) this.wake(j);
let wi = this.asleep[i] ? 0 : this.invMass[i];
let wj = this.asleep[j] ? 0 : this.invMass[j];
if (shock && ny !== 0) {
  if (ny > 0) wi = 0; // i is below j
  else wj = 0;
}
const w = wi + wj;
if (w === 0) return;
```

`vn` is how fast the boxes approach each other along the normal (negative means closing in). A sleeping box acts as an immovable object, unless it's hit faster than `WAKE_SPEED`, which wakes it.

Then **shock propagation**, on the last pass only: in a vertical contact, the lower box gets an inverse mass of 0, as if it were the floor. Combined with the bottom-up order, the floor's support travels from the bottom of a stack to the top in one sweep.

> Why this was needed: without it, every box in a tall stack shared the same downward velocity. Between neighbours the relative velocity was zero, so no impulse ever cancelled it. A 3,000-box pile stayed awake and jittering forever; with it, the pile is asleep within about two seconds.

Pushing apart

src/physics/AabbWorld.ts · lines 351–356

```
// Push apart, split by inverse mass (the lighter box moves more).
const push = (Math.max(depth - SLOP, 0) * CORRECTION) / w;
px[i] -= nx * push * wi; py[i] -= ny * push * wi; pz[i] -= nz * push * wi;
px[j] += nx * push * wj; py[j] += ny * push * wj; pz[j] += nz * push * wj;

if (vn >= 0) return; // already separating
```

Remove most (80%) of the overlap beyond the allowed slop, shared in proportion to inverse mass, so a light box moves more than a heavy one and a sleeping or shocked box doesn't move at all. If they're already moving apart, there's nothing more to do.

Normal impulse

src/physics/AabbWorld.ts · lines 358–362

```
// Normal impulse: cancel the closing speed (plus a little bounce for real impacts).
const e = vn < -BOUNCE_THRESHOLD ? RESTITUTION : 0;
const jn = (-(1 + e) * vn) / w;
vx[i] -= nx * jn * wi; vy[i] -= ny * jn * wi; vz[i] -= nz * jn * wi;
vx[j] += nx * jn * wj; vy[j] += ny * jn * wj; vz[j] += nz * jn * wj;
```

An **impulse** is an instant change in momentum. This one is sized to exactly cancel the closing speed (or reverse a little of it, if the impact is hard enough to bounce), split between the two boxes by inverse mass. Equal and opposite: one box loses what the other gains, as Newton's third law says.

Friction

src/physics/AabbWorld.ts · lines 364–372

```
  // Friction: oppose sliding, but no more than FRICTION × the normal impulse (Coulomb).
  let tx = rvx - vn * nx, ty = rvy - vn * ny, tz = rvz - vn * nz;
  const slide = Math.sqrt(tx * tx + ty * ty + tz * tz);
  if (slide < 1e-9) return;
  const jt = Math.min(slide / w, FRICTION * jn);
  tx /= slide; ty /= slide; tz /= slide;
  vx[i] += tx * jt * wi; vy[i] += ty * jt * wi; vz[i] += tz * jt * wi;
  vx[j] -= tx * jt * wj; vy[j] -= ty * jt * wj; vz[j] -= tz * jt * wj;
}
```

`t` is the sliding part of the relative velocity (with the normal part removed). Friction pushes against the slide, but **Coulomb's law** caps it at `FRICTION` × the normal impulse: pressing harder allows more grip. If the cap isn't reached, the slide stops completely (static friction); if it is, the box keeps sliding, slower (kinetic friction).

Boxes against statics: the conveyor

src/physics/AabbWorld.ts · lines 374–412

```
/** Box vs every static collider. Statics never move, so the box takes the whole correction. */
private solveStatics(i: number, first: boolean): void {
  const { px, py, pz, vx, vy, vz } = this;
  for (const s of this.statics) {
    const dx = px[i] - s.cx, dy = py[i] - s.cy, dz = pz[i] - s.cz;
    const ox = this.hx[i] + s.hx - Math.abs(dx);
    if (ox <= 0) continue;
```

```
    const oy = this.hy[i] + s.hy - Math.abs(dy);
    if (oy <= 0) continue;
    const oz = this.hz[i] + s.hz - Math.abs(dz);
    if (oz <= 0) continue;
    if (first) this.stats.contacts++;
    if (s.conveyor) this.onBelt[i] = 1;

    let nx = 0, ny = 0, nz = 0, depth: number;
    if (ox <= oy && ox <= oz) { depth = ox; nx = dx < 0 ? -1 : 1; }
    else if (oy <= oz) { depth = oy; ny = dy < 0 ? -1 : 1; }
    else { depth = oz; nz = dz < 0 ? -1 : 1; }

    const push = Math.max(depth - SLOP, 0) * CORRECTION;
    px[i] += nx * push; py[i] += ny * push; pz[i] += nz * push;

    // Velocity relative to the surface. The belt's surface moves, so friction drags boxes
along.
    const rvx = vx[i] - (s.conveyor ? this.speed : 0), rvy = vy[i], rvz = vz[i];
    const vn = rvx * nx + rvy * ny + rvz * nz;
    if (vn >= 0) continue;

    const e = vn < -BOUNCE_THRESHOLD ? RESTITUTION : 0;
    const dvn = -(1 + e) * vn;
    vx[i] += nx * dvn; vy[i] += ny * dvn; vz[i] += nz * dvn;

    let tx = rvx - vn * nx, ty = rvy - vn * ny, tz = rvz - vn * nz;
    const slide = Math.sqrt(tx * tx + ty * ty + tz * tz);
    if (slide < 1e-9) continue;
    const dvt = Math.min(slide, FRICTION * dvn);
    tx /= slide; ty /= slide; tz /= slide;
    vx[i] -= tx * dvt; vy[i] -= ty * dvt; vz[i] -= tz * dvt;
  }
}
```

The same steps, simplified: a static never moves, so the box takes the whole correction and impulses become plain velocity changes.

The one difference is line 397. Relative velocity is measured against the **surface**, and the belt's surface moves at `speed`. A box sitting still on a running belt is sliding backwards relative to it, so friction pushes it forward until it matches the belt. The belt collider itself never moves: there's no special conveyor code anywhere else, and jams and slipping just happen.

4. Sleeping

src/physics/AabbWorld.ts · lines 414–437

```
// ---- 4. Sleeping ----

private updateSleep(dt: number, pool: SlotPool): void {
  const { vx, vy, vz, asleep, sleepTime } = this;
  const { dense, count } = pool;
  let awake = 0;
  for (let k = 0; k < count; k++) {
    const i = dense[k];
    if (asleep[i]) continue;
    const beltHolds = this.onBelt[i] === 1 && this.speed !== 0; // jammed on a running belt:
stay awake
    if (!beltHolds && vx[i] * vx[i] + vy[i] * vy[i] + vz[i] * vz[i] < SLEEP_SPEED_SQ) {
      sleepTime[i] += dt;
      if (sleepTime[i] >= SLEEP_TIME) {
        asleep[i] = 1;
        vx[i] = vy[i] = vz[i] = 0;
        continue;
      }
    } else {
      sleepTime[i] = 0;
    }
    awake++;
  }
  this.stats.awake = awake;
}
```

A box that stays slower than 6 cm/s for 0.4 s goes to sleep: its velocity is zeroed and it's skipped by integration, solving and (for sleeper–sleeper pairs) the broad phase. A settled pile then costs almost nothing.

The exception is a box touching a **running** belt. It might be jammed and motionless, but the belt is still pulling on it, so it has to stay simulated.

Waking whole piles

src/physics/AabbWorld.ts · lines 439–460

```
/**
 * Wakes a body and, transitively, every sleeping body touching it: the whole
 * resting pile it belongs to (an "island"). Otherwise a box could stay asleep
 * floating in mid-air after the box under it was knocked away.
 */
private wake(start: number): void {
  if (this.asleep[start] !== 1) return;
  this.asleep[start] = 2;
  this.wakeStack[0] = start;
  this.drainWake(1);
}

/** Wakes everything on the stack, pushing each one's sleeping neighbors in turn (depth-first
flood fill). */
private drainWake(top: number): void {
  const stack = this.wakeStack;
  while (top > 0) {
    const i = stack[--top];
    this.asleep[i] = 0;
    this.sleepTime[i] = 0;
    top = this.pushSleepingNeighbors(i, top);
  }
}
```

Waking one box isn't enough. If a box is knocked out of a pile, the sleeping boxes on top of it would hang in mid-air, because sleeping boxes aren't simulated. So waking spreads to every sleeping box touching the woken one, and then to those touching them, until the whole connected pile (an **island**) is awake. Engines like Box2D and PhysX do the same.

It's a depth-first flood fill with an explicit stack array instead of recursion, so a 3,000-box pile can't overflow the call stack. While queued, a box is marked `2`, so it can't be pushed twice.

Finding sleeping neighbors

src/physics/AabbWorld.ts · lines 466–498

```
private pushSleepingNeighbors(i: number, top: number): number {
  const stack = this.wakeStack;
  const near = (j: number) =>
    j !== i &&
```

```
    this.alive[j] === 1 &&
    this.asleep[j] === 1 &&
    Math.abs(this.px[j] - this.px[i]) < this.hx[i] + this.hx[j] + WAKE_MARGIN &&
    Math.abs(this.py[j] - this.py[i]) < this.hy[i] + this.hy[j] + WAKE_MARGIN &&
    Math.abs(this.pz[j] - this.pz[i]) < this.hz[i] + this.hz[j] + WAKE_MARGIN;
  const mark = (j: number) => {
    this.asleep[j] = 2;
    stack[top++] = j;
  };

  if (this.hashBuilt) {
    // Sleeping bodies haven't moved since the hash was built, so their cells are still right.
    const h = this.hash;
    const x = h.cell(this.px[i]);
    const y = h.cell(this.py[i]);
    const z = h.cell(this.pz[i]);
    for (let dx = -1; dx <= 1; dx++)
      for (let dy = -1; dy <= 1; dy++)
        for (let dz = -1; dz <= 1; dz++) {
          for (let j = h.head[h.bucket(x + dx, y + dy, z + dz)]; j !== -1; j = h.next[j]) {
            if (h.cx[j] === x + dx && h.cy[j] === y + dy && h.cz[j] === z + dz && near(j))
mark(j);
          }
        }
  } else if (this.lastPool) {
    const { dense, count } = this.lastPool;
    for (let k = 0; k < count; k++) if (near(dense[k])) mark(dense[k]);
  }
  return top;
}
```

Uses the spatial hash from the last step to look in the 27 cells around the box. That's still accurate, because sleeping boxes haven't moved since the grid was built. Without the hash (the all-pairs mode), it scans every live box instead.

The two small arrow functions (`near`, `mark`) keep the loops readable. They're created per call, which is fine here: waking happens a few times a second, not millions of times a step.

GOING DEEPER

What a full engine adds

- **Rotation**: orientation, angular velocity, inertia tensors, and impulses applied at contact points so they create torque.
- **Oriented collision tests**: separating-axis tests (SAT) for boxes, GJK/EPA for general convex shapes, and contact **manifolds** of up to four points per pair, so a box can rest flat instead of rocking on one corner.
- **Warm starting** and caching of contacts between steps.
- **Continuous collision detection** for fast, thin objects, instead of capping speed and thickening the floor.
- **Joints**: hinges, sliders, motors, all as more constraints in the same solver.

PART 3.7 · `src/physics/CannonWorld.ts`

The cannon-es adapter

The same interface, implemented with a real rigid-body engine: rotation, tumbling, and constraint-based contacts.

Setup

src/physics/CannonWorld.ts · lines 29–38

```
constructor(capacity: number, statics: readonly StaticCollider[] = STATICS) {
  this.world = new CANNON.World({ gravity: new CANNON.Vec3(0, GRAVITY, 0), allowSleep: true });
  this.world.broadphase = new CANNON.SAPBroadphase(this.world);
  (this.world.solver as CANNON.GSSolver).iterations = 8;
  this.world.defaultContactMaterial.friction = 0.5;
  this.world.defaultContactMaterial.restitution = 0.1;

  this.bodies = new Array<CANNON.Body | null>(capacity).fill(null);
  this.prev = new Float64Array(capacity * 7);
```

A cannon-es `World` with gravity and sleeping on. **Sweep-and-prune** (SAP) is cannon's broad phase: it sorts bodies along an axis and only compares neighbours, a different route to the same O(n)-ish goal as the spatial hash. Friction and bounciness match the custom engine so the comparison is fair.

Statics and the kinematic belt

src/physics/CannonWorld.ts · lines 39–59

```
  let belt: CANNON.Body | null = null;
  for (const s of statics) {
    const body = new CANNON.Body({
      mass: 0,
      // The belt is kinematic: it has a velocity, so friction carries boxes along,
      // but we put it back in place every step so it never actually moves.
      type: s.conveyor ? CANNON.Body.KINEMATIC : CANNON.Body.STATIC,
      shape: new CANNON.Box(new CANNON.Vec3((s.max.x - s.min.x) / 2, (s.max.y - s.min.y) / 2,
(s.max.z - s.min.z) / 2)),
      position: new CANNON.Vec3((s.min.x + s.max.x) / 2, (s.min.y + s.max.y) / 2, (s.min.z +
s.max.z) / 2),
    });
    if (s.conveyor) {
      body.allowSleep = false;
      belt = body;
    }
    this.world.addBody(body);
  }
  if (!belt) throw new Error("The layout has no conveyor belt");
  this.belt = belt;
  this.beltHome = belt.position.clone();
  this.beltSpeed = 0.8;
}
```

Each static collider becomes a mass-zero cannon body with a box shape (cannon also takes half extents). The belt is **kinematic**: it has a velocity but isn't pushed around by anything. cannon's friction uses that velocity, so boxes on it are carried along, which is the same surface-velocity idea as the custom engine. The step function puts it back in place each step, so it never actually travels.

Adding boxes

src/physics/CannonWorld.ts · lines 70–89

```
add(slot: number, d: BodyDesc): void {
  const key = `${d.hx}|${d.hy}|${d.hz}`;
  let shape = this.shapes.get(key);
  if (!shape) {
    shape = new CANNON.Box(new CANNON.Vec3(d.hx, d.hy, d.hz));
    this.shapes.set(key, shape);
  }
  const body = new CANNON.Body({
    mass: 8 * d.hx * d.hy * d.hz * DENSITY,
    shape,
    position: new CANNON.Vec3(d.x, d.y, d.z),
    sleepSpeedLimit: 0.06,
    sleepTimeLimit: 0.4,
    linearDamping: 0.01,
    angularDamping: 0.05,
  });
  this.world.addBody(body);
  this.bodies[slot] = body;
  this.writePrev(slot, body);
}
```

Shapes are cached by size in a `Map`, so all boxes of one size share a `CANNON.Box`. Bodies are still created per spawn, because cannon owns its body objects. That allocation is part of what the comparison shows.

Removing, and waking

src/physics/CannonWorld.ts · lines 91–99

```
remove(slot: number): void {
  const body = this.bodies[slot];
  if (!body) return;
  this.world.removeBody(body);
  this.bodies[slot] = null;
  // cannon-es doesn't wake bodies resting on a removed one, so do it ourselves,
  // spreading through anything touching (the same island idea as the custom engine).
  this.wakeAround(body);
}
```

src/physics/CannonWorld.ts · lines 168–185

```
/** Wakes sleeping bodies whose bounding spheres touch `start`'s, and so on outward. */
private wakeAround(start: CANNON.Body): void {
  const pool = this.lastPool;
  if (!pool) return;
  const queue: CANNON.Body[] = [start];
  while (queue.length > 0) {
    const a = queue.pop()!;
    for (let k = 0; k < pool.count; k++) {
      const b = this.bodies[pool.dense[k]];
      if (!b || b === a || b.sleepState !== CANNON.Body.SLEEPING) continue;
      const reach = a.boundingRadius + b.boundingRadius + WAKE_MARGIN;
      if (a.position.distanceSquared(b.position) < reach * reach) {
        b.wakeUp();
        queue.push(b);
      }
    }
  }
}
```

cannon-es doesn't wake bodies that were resting on a removed one, so the adapter does it itself, with the same island idea: wake sleeping bodies whose bounding spheres touch, then theirs, and so on. Bodies can be rotated here, so it uses bounding spheres (`boundingRadius`) rather than axis-aligned boxes.

Stepping and interpolation

src/physics/CannonWorld.ts · lines 109–122

```
step(dt: number, pool: SlotPool): void {
  this.lastPool = pool;
  this.belt.position.copy(this.beltHome);
  this.world.step(dt); // one fixed step; our FixedStepper does the accumulating

  this.stats.contacts = this.world.contacts.length;
  let awake = 0;
  const { dense, count } = pool;
  for (let k = 0; k < count; k++) {
    const body = this.bodies[dense[k]];
    if (body && body.sleepState !== CANNON.Body.SLEEPING) awake++;
  }
  this.stats.awake = awake;
}
```

src/physics/CannonWorld.ts · lines 131–151

```
readTransform(slot: number, alpha: number, pos: V3, quat: Quat): void {
  const body = this.bodies[slot]!;
  const o = slot * 7;
  const prev = this.prev;
  pos.x = prev[o] + (body.position.x - prev[o]) * alpha;
  pos.y = prev[o + 1] + (body.position.y - prev[o + 1]) * alpha;
  pos.z = prev[o + 2] + (body.position.z - prev[o + 2]) * alpha;

  // Normalized lerp between the two rotations: close enough to slerp over one 1/120 s step.
  const q = body.quaternion;
  const sign = prev[o + 3] * q.x + prev[o + 4] * q.y + prev[o + 5] * q.z + prev[o + 6] * q.w <
0 ? -1 : 1;
  const x = prev[o + 3] + (sign * q.x - prev[o + 3]) * alpha;
  const y = prev[o + 4] + (sign * q.y - prev[o + 4]) * alpha;
  const z = prev[o + 5] + (sign * q.z - prev[o + 5]) * alpha;
  const w = prev[o + 6] + (sign * q.w - prev[o + 6]) * alpha;
  const len = Math.hypot(x, y, z, w) || 1;
  quat.x = x / len;
  quat.y = y / len;
  quat.z = z / len;
  quat.w = w / len;
}
```

`world.step(dt)` with a single argument runs exactly one fixed step; the shared `FixedStepper` does the accumulating, exactly as for the custom engine.

Positions interpolate as before. Rotations use a **normalized lerp** of the two quaternions: blend the four numbers, then rescale to length 1. If the dot product is negative, one quaternion is flipped first. `q` and `−q` are the same rotation, and without the flip the blend would spin the long way round.

GOING DEEPER

Reading the comparison fairly

cannon-es does more work per body than the custom engine: rotation, oriented contacts, an object per body, and a general solver. A side-by-side comparison measures the **cost of generality** in this specific scene. It isn't evidence that one engine is “better”: change the scene to tumbling objects and only one of them can represent it at all.

PART 3.8 · `src/sim/BoxFlow.ts`

Spawning and recycling

Decides which boxes exist. Knows nothing about rendering, and works with either engine.

Per-frame update

src/sim/BoxFlow.ts · lines 36–54

```
/** Call once per frame. */
update(dt: number): void {
  this.clock += dt;
  while (this.pool.count > this.target) this.retireOldest();

  if (this.target > 0) {
    // Accumulate fractional boxes so e.g. 6/s spawns evenly whatever the frame rate.
    this.spawnDebt += this.rate * dt;
    while (this.spawnDebt >= 1) {
      this.spawnDebt -= 1;
      if (this.pool.count >= this.target) this.retireOldest(); // recycle: oldest out, new one
in
      this.spawn(SPAWN.x + (this.random() - 0.5) * 0.04, SPAWN.y, (this.random() * 2 - 1) *
SPAWN.zJitter);
    }
  } else {
    this.spawnDebt = 0;
  }

  this.retireOutOfBounds();
}
```

If the target was lowered, the oldest boxes are removed until it's met. Then spawning uses a **debt** counter: at 6 boxes/s and 60 fps, each frame adds 0.1, and a box appears every time it passes 1. That keeps the rate exact at any frame rate.

At the target, each new box first retires the oldest one, so the scene keeps flowing instead of stopping when full.

The stress-test burst

src/sim/BoxFlow.ts · lines 56–73

```
/** Drops `n` boxes at once in layers above the bin and the floor around it. Returns the new
target. */
burst(n: number): number {
  const a = BURST_AREA;
  const cols = Math.floor((a.maxX - a.minX) / a.spacing) + 1;
  const rows = Math.floor((a.maxZ - a.minZ) / a.spacing) + 1;
  const room = Math.min(n, this.pool.capacity - this.pool.count);
  for (let k = 0; k < room; k++) {
    const layer = Math.floor(k / (cols * rows));
    const inLayer = k % (cols * rows);
    this.spawn(
      a.minX + (inLayer % cols) * a.spacing,
      a.baseY + layer * a.spacing,
      a.minZ + Math.floor(inLayer / cols) * a.spacing,
    );
  }
  this.target = Math.max(this.target, this.pool.count);
  return this.target;
}
```

Lays `n` boxes out in a grid: `cols × rows` per layer, then more layers above. `%` and `Math.floor(k / …)` turn a running count `k` into a column, row and layer. It raises the target too, so the burst isn't immediately recycled away.

Switching engines mid-run

src/sim/BoxFlow.ts · lines 80–91

```
/** Moves every live box into another engine, at its current position, so engines can be
switched mid-run. */
switchBackend(next: PhysicsBackend): void {
  const { dense, count } = this.pool;
  next.beltSpeed = this.backend.beltSpeed;
  for (let k = 0; k < count; k++) {
    const slot = dense[k];
    this.backend.getPosition(slot, this.tmp);
    const h = SIZE_PRESETS[this.preset[slot]];
    next.add(slot, { x: this.tmp.x, y: this.tmp.y, z: this.tmp.z, hx: h.x, hy: h.y, hz: h.z });
  }
  this.backend = next;
}
```

Copies every live box into the new engine at its current position, keeping its slot number, so the renderer and the pool don't notice. Velocities and rotations aren't carried over, which is fine for a demo.

Spawning and retiring

src/sim/BoxFlow.ts · lines 93–106

```
private spawn(x: number, y: number, z: number): void {
  const slot = this.pool.alloc();
  if (slot < 0) return; // pool full
  const p = Math.floor(this.random() * SIZE_PRESETS.length);
  const h = SIZE_PRESETS[p];
  this.preset[slot] = p;
  this.spawnedAt[slot] = this.clock;
  this.backend.add(slot, { x, y, z, hx: h.x, hy: h.y, hz: h.z });
}

private retire(slot: number): void {
  this.backend.remove(slot);
  this.pool.release(slot);
}
```

src/sim/BoxFlow.ts · lines 108–126

```
/** O(n) scan; at most a few dozen times a second, so simpler than keeping a queue. */
private retireOldest(): void {
  const { dense, count } = this.pool;
  if (count === 0) return;
  let oldest = dense[0];
  for (let k = 1; k < count; k++) if (this.spawnedAt[dense[k]] < this.spawnedAt[oldest]) oldest
= dense[k];
  this.retire(oldest);
}

private retireOutOfBounds(): void {
  const { dense } = this.pool;
  // Walk backwards: release() swaps the last live slot into the hole, and we've already
checked that one.
  for (let k = this.pool.count - 1; k >= 0; k--) {
    const slot = dense[k];
    this.backend.getPosition(slot, this.tmp);
    const p = this.tmp;
    if (p.y < BOUNDS.minY || Math.abs(p.x) > BOUNDS.maxAbsXZ || Math.abs(p.z) >
BOUNDS.maxAbsXZ) this.retire(slot);
  }
}
```

`retireOldest` is a simple O(n) scan for the smallest spawn time. It runs a few dozen times a second at most, so the simplicity wins over maintaining a queue. Out-of-bounds removal walks the list backwards, because swap-remove moves the last element into the hole: walking backwards, that element has already been checked.

Repeatable randomness

src/sim/BoxFlow.ts · lines 128–132

```
/** Small seeded generator, so runs (and benchmarks) are repeatable. */
private random(): number {
  this.seed = (Math.imul(this.seed, 1664525) + 1013904223) >>> 0;
  return this.seed / 2 ** 32;
}
```

A tiny **linear congruential generator** instead of `Math.random()`, seeded with a fixed number, so every run and every benchmark spawns the same sequence of boxes. `>>> 0` keeps the result an unsigned 32-bit integer; dividing by 2^{32} maps it into [0, 1).

PART 3.9 · `src/render/BoxRenderer.ts`

Drawing thousands of boxes

The rendering half of the performance story: one draw call for every box, instead of one each.

Two ways to draw

src/render/BoxRenderer.ts · lines 10–39

```
/**
 * Draws every box, two ways:
 *
 * - "instanced": one InstancedMesh. All boxes share one geometry and one material,
 *   and each box is just a matrix and a color in a GPU buffer, so 3,000 boxes are a
 *   single draw call (plus one for the shadow map).
 * - "meshes": one THREE.Mesh per box, the straightforward way. Same picture, but
 *   one draw call per box per pass. Here for comparison.
 *
 * Both reuse their objects frame to frame; nothing is allocated per frame.
 */
export class BoxRenderer {
  readonly group = new THREE.Group();
  mode: RenderMode = "instanced";
  tintSleeping = false;

  private readonly geometry = new THREE.BoxGeometry(1, 1, 1);
  private readonly instanced: THREE.InstancedMesh;
  private readonly colors = PALETTE.map((c) => new THREE.Color(c));
  private readonly sleepingColor = new THREE.Color(SLEEPING);
  private readonly materials = PALETTE.map((c) => new THREE.MeshStandardMaterial({ color: c,
roughness: 0.7 }));
  private readonly sleepingMaterial = new THREE.MeshStandardMaterial({ color: SLEEPING,
roughness: 0.7 });
  private readonly meshes: THREE.Mesh[] = [];
  private meshesShown = 0;

  // Scratch objects, reused every frame.
  private readonly matrix = new THREE.Matrix4();
  private readonly pos = new THREE.Vector3();
  private readonly quat = new THREE.Quaternion();
  private readonly scale = new THREE.Vector3();
```

A **draw call** is one request from the CPU to the GPU: “draw this geometry with this material”. Each has a fixed overhead, so thousands of tiny ones are slow even when each draws only 12 triangles.

An `InstancedMesh` draws one geometry many times in a single call, reading each copy's matrix (position, rotation, scale) and color from a buffer on the GPU. The scratch `matrix`, `pos`, `quat` and `scale` objects are reused for every box, every frame.

Setting up the instanced mesh

src/render/BoxRenderer.ts · lines 41–57

```
constructor(capacity: number) {
  // White base color: the per-instance color multiplies it.
  this.instanced = new THREE.InstancedMesh(
    this.geometry,
    new THREE.MeshStandardMaterial({ color: 0xffffff, roughness: 0.7 }),
    capacity,
  );
  this.instanced.instanceMatrix.setUsage(THREE.DynamicDrawUsage);
  this.instanced.setColorAt(0, this.colors[0]); // creates the instanceColor buffer
  this.instanced.instanceColor!.setUsage(THREE.DynamicDrawUsage);
  this.instanced.count = 0;
  this.instanced.castShadow = true;
  this.instanced.receiveShadow = true;
  // Instances move every frame, so a bounding sphere computed once would soon be wrong.
  this.instanced.frustumCulled = false;
  this.group.add(this.instanced);
}
```

One 1 m cube, scaled per box. The material is white because each instance's color multiplies it. `DynamicDrawUsage` tells the GPU driver these buffers change every frame. Calling `setColorAt` once creates the color buffer.

Frustum culling normally skips objects outside the camera's view, using a bounding sphere. For an instanced mesh that sphere is computed once and goes stale as boxes move, so culling is turned off rather than risk boxes vanishing.

The instanced path

src/render/BoxRenderer.ts · lines 59–77

```
update(flow: BoxFlow, alpha: number): void {
  const { dense, count } = flow.pool;
  const backend = flow.backend;

  if (this.mode === "instanced") {
    this.hideMeshes(0);
    this.instanced.visible = true;
    for (let k = 0; k < count; k++) {
      const slot = dense[k];
      this.composeMatrix(flow, slot, alpha);
      this.instanced.setMatrixAt(k, this.matrix);
      const sleeping = this.tintSleeping && backend.isAsleep(slot);
      this.instanced.setColorAt(k, sleeping ? this.sleepingColor :
this.colors[flow.preset[slot]]);
    }
    this.instanced.count = count;
    this.instanced.instanceMatrix.needsUpdate = true;
    this.instanced.instanceColor!.needsUpdate = true;
    return;
  }
```

For each live box: get its interpolated transform from whichever engine is running, build its matrix, and write the matrix and color into position `k`. Setting `count` tells Three.js to draw only the first `count` instances; `needsUpdate` tells it to upload the changed buffers. Coloring every box every frame is cheap, and it means swap-removal reordering never shows.

The one-mesh-per-box path

src/render/BoxRenderer.ts · lines 79–91

```
  this.instanced.visible = false;
  for (let k = 0; k < count; k++) {
    const slot = dense[k];
    const mesh = this.meshes[k] ?? this.createMesh();
    const sleeping = this.tintSleeping && backend.isAsleep(slot);
    mesh.material = sleeping ? this.sleepingMaterial : this.materials[flow.preset[slot]];
    backend.readTransform(slot, alpha, mesh.position, mesh.quaternion);
    const h = SIZE_PRESETS[flow.preset[slot]];
    mesh.scale.set(2 * h.x, 2 * h.y, 2 * h.z);
    mesh.visible = true;
  }
  this.hideMeshes(count);
}
```

src/render/BoxRenderer.ts · lines 100–113

```
private createMesh(): THREE.Mesh {
  const mesh = new THREE.Mesh(this.geometry, this.materials[0]);
  mesh.castShadow = true;
  mesh.receiveShadow = true;
  this.meshes.push(mesh);
  this.group.add(mesh);
  return mesh;
}

/** Hides pooled meshes from index `from` up to however many were shown last frame. */
private hideMeshes(from: number): void {
  for (let k = from; k < this.meshesShown; k++) this.meshes[k].visible = false;
  this.meshesShown = from;
}
```

The straightforward way, kept for comparison. Meshes are pooled too: created only when more are needed than ever before, and hidden rather than deleted when not. `readTransform` writes straight into each mesh's `position` and `quaternion`, since they're compatible with the engine's plain objects.

Unticking **Instancing** with ~500 boxes visible took the scene from 27 draw calls to 484.

GOING DEEPER

What the GPU actually receives

`instanceMatrix` is a buffer attribute with a divisor of 1: the vertex shader reads a new matrix per instance instead of per vertex. The whole pile is one `drawElementsInstanced` call. What remains per frame is uploading the changed buffers: at the 4,096-box capacity that's 256 KB of matrices if every one changed. Beyond that, the next steps are writing matrices straight into the array rather than composing them, and eventually moving the simulation to the GPU with compute shaders.

PART 3.10 · `src/render/Workcell.ts`

The conveyor and bin

Visuals built from the same collider list the physics uses, plus a scrolling texture to show the belt moving.

Drawing the colliders

src/render/Workcell.ts · lines 11–31

```
export function buildWorkcell(scene: THREE.Scene) {
  const steel = new THREE.MeshStandardMaterial({ color: 0x9aa3ae, metalness: 0.7, roughness:
0.35 });
  const dark = new THREE.MeshStandardMaterial({ color: 0x2a2e35, metalness: 0.4, roughness: 0.6
});
  const binMat = new THREE.MeshStandardMaterial({ color: 0x3b6e8f, metalness: 0.3, roughness:
0.6 });

  const beltTexture = makeBeltTexture();
  beltTexture.repeat.set((BELT.maxX - BELT.minX) / STRIPE_PERIOD, 1);
  const beltTop = new THREE.MeshStandardMaterial({ map: beltTexture, roughness: 0.85 });

  for (const s of STATICS) {
    if (s.visible === false) continue;
    const size = new THREE.Vector3(s.max.x - s.min.x, s.max.y - s.min.y, s.max.z - s.min.z);
    // BoxGeometry face order is +x, -x, +y, -y, +z, -z: only the belt's top face gets the
texture.
    const material = s.conveyor ? [dark, dark, beltTop, dark, dark, dark] :
s.name.startsWith("bin") ? binMat : steel;
    const mesh = new THREE.Mesh(new THREE.BoxGeometry(size.x, size.y, size.z), material);
    mesh.position.set((s.min.x + s.max.x) / 2, (s.min.y + s.max.y) / 2, (s.min.z + s.max.z) /
2);
    mesh.castShadow = true;
    mesh.receiveShadow = true;
    mesh.name = s.name;
    scene.add(mesh);
  }
```

Every visible static becomes a box mesh of exactly its size and position, so the picture can't drift from the physics. A box mesh can take an array of six materials, one per face (+x, −x, +y, −y, +z, −z), so only the belt's top face gets the striped texture.

A belt that looks like it moves

src/render/Workcell.ts · lines 73–78

```
return {
  update(dt: number, beltSpeed: number) {
    // Scrolling the texture makes the belt look like it moves; the physics belt never moves.
    beltTexture.offset.x = (beltTexture.offset.x - (beltSpeed * dt) / STRIPE_PERIOD) % 1;
  },
};
```

src/render/Workcell.ts · lines 83–101

```
  const canvas = document.createElement("canvas");
  canvas.width = 64;
  canvas.height = 128;
  const g = canvas.getContext("2d")!;
  g.fillStyle = "#23272e";
  g.fillRect(0, 0, 64, 128);
  g.strokeStyle = "#3a414d";
  g.lineWidth = 10;
  g.beginPath();
  g.moveTo(10, 0);
  g.lineTo(44, 64);
  g.lineTo(10, 128);
  g.stroke();
  const texture = new THREE.CanvasTexture(canvas);
  texture.wrapS = texture.wrapT = THREE.RepeatWrapping;
  texture.colorSpace = THREE.SRGBColorSpace;
  texture.anisotropy = 4;
  return texture;
}
```

The physics belt never moves, so neither does its mesh. Instead the texture slides: shifting `offset.x` by speed × time ÷ stripe spacing each frame moves the chevrons at exactly the belt speed. `% 1` keeps the number small, since the texture repeats.

The chevron is drawn on an off-screen `<canvas>` and used as a texture, so there's no image file to load. `RepeatWrapping` tiles it along the belt.

PART 3.11 · `src/main.ts`

The frame loop

Wires everything together and measures it.

Setup

src/main.ts · lines 14–24

```
const canvas = document.getElementById("scene") as HTMLCanvasElement;
const { renderer, scene, camera, controls } = createStage(canvas);
const workcell = buildWorkcell(scene);

const boxes = new BoxRenderer(MAX_BODIES);
scene.add(boxes.group);

const flow = new BoxFlow(new AabbWorld(MAX_BODIES));
const stepper = new FixedStepper(FIXED_STEP, MAX_SUBSTEPS);
let spatialHash = true;
let switching = false;
```

Scene, conveyor visuals, box renderer, the flow (starting on the custom engine), and the fixed stepper. `MAX_BODIES` is passed everywhere so all the arrays match.

Controls

src/main.ts · lines 26–44

```
const hud = createHud({
  onEngine: (kind) => void setEngine(kind),
  onSpatialHash: (on) => {
    spatialHash = on;
    if (flow.backend instanceof AabbWorld) flow.backend.useSpatialHash = on;
  },
  onInstancing: (on) => (boxes.mode = on ? "instanced" : "meshes"),
  onTintSleeping: (on) => (boxes.tintSleeping = on),
  onTarget: (n) => (flow.target = n),
  onRate: (r) => (flow.rate = r),
  onBeltSpeed: (v) => (flow.backend.beltSpeed = v),
  onBurst: () => hud.setTarget(flow.burst(BURST_SIZE)),
  onClear: () => {
    flow.clear();
    flow.target = 0;
    hud.setTarget(0);
  },
});
hud.setEngine("custom");
```

Each panel control maps to one assignment. `void setEngine(kind)` calls an async function without waiting for it; `void` says that's deliberate. **Drop 500** returns the new target so the slider can show it.

Switching engine, with cannon-es on demand

src/main.ts · lines 46–71

```
/**
 * Swaps the physics engine mid-run, carrying every box over at its current position.
 * cannon-es is loaded on demand (a separate chunk), so the page starts without it.
 */
async function setEngine(kind: EngineKind) {
  if (switching) return;
  switching = true;
  try {
    let next: PhysicsBackend;
    if (kind === "cannon") {
      const { CannonWorld } = await import("./physics/CannonWorld");
      next = new CannonWorld(MAX_BODIES);
```

```
    } else {
      const custom = new AabbWorld(MAX_BODIES);
      custom.useSpatialHash = spatialHash;
      next = custom;
    }
    flow.switchBackend(next);
    hud.setEngine(kind);
  } catch (err) {
    console.error(err);
    hud.showMessage("Couldn't load cannon-es. Run npm install, then reload.");
  } finally {
    switching = false;
  }
}
```

`await import("./physics/CannonWorld")` is a **dynamic import**: Vite puts cannon-es in a separate file that's only downloaded the first time someone picks it, so the page starts faster. `switching` stops a double click from starting two switches. If loading fails, the panel says so instead of breaking.

Each frame

src/main.ts · lines 78–96

```
renderer.setAnimationLoop((now) => {
  const frameMs = now - last;
  last = now;
  const dt = Math.min(frameMs / 1000, 0.25); // after a hidden tab, don't try to simulate
minutes

  flow.update(dt);

  // Physics: fixed 1/120 s steps, timed so the panel can show what physics costs per frame.
  const t0 = performance.now();
  const steps = stepper.advance(dt, () => {
    flow.backend.savePrevious(flow.pool);
    flow.backend.step(FIXED_STEP, flow.pool);
  });
  const physicsMs = performance.now() - t0;

  workcell.update(dt, flow.backend.beltSpeed);
  boxes.update(flow, stepper.alpha); // draw between the last two physics states
  controls.update();
  renderer.render(scene, camera);
```

In order: work out the frame time (capped at 0.25 s so a tab returning from the background doesn't try to simulate minutes), spawn or retire boxes, run the fixed physics steps (timed with `performance.now()`), scroll the belt, update the boxes using `alpha`, and draw. Each physics step first saves the previous state, so interpolation always has the last two.

Stats without overhead

src/main.ts · lines 98–118

```
hud.pushFrame(frameMs);
sample.time += dt;
sample.frames++;
sample.physics += physicsMs;
sample.steps += steps;
if (sample.time >= STATS_INTERVAL) {
  const s = flow.backend.stats;
  hud.showStats({
    fps: sample.frames / sample.time,
    frameMs: (sample.time * 1000) / sample.frames,
    physicsMs: sample.physics / sample.frames,
    stepsPerFrame: sample.steps / sample.frames,
    boxes: flow.count,
    awake: s.awake,
    drawCalls: renderer.info.render.calls,
    pairsTested: s.pairsTested,
    contacts: s.contacts,
    droppedMs: stepper.droppedTime * 1000,
  });
  sample = { time: 0, frames: 0, physics: 0, steps: 0 };
}
```

The graph gets every frame's time. The numbers are averaged over a quarter second and written only then, because updating text 144 times a second is wasted work nobody can read. Draw calls come from `renderer.info`, which Three.js fills in during `render()`.

PART 3.12 · `src/ui/hud.ts`

The frame-time graph

The panel is plain DOM; the one interesting piece is the graph.

A ring buffer, drawn every frame

src/ui/hud.ts · lines 87–116

```
// Frame-time graph: a ring buffer of the last 120 frames, drawn every frame.
const graph = byId<HTMLCanvasElement>("graph");
```

```
const g = graph.getContext("2d")!;
const history = new Float32Array(120);
let head = 0;

return {
  /** Every frame: feeds the graph. */
  pushFrame(ms: number) {
    history[head] = ms;
    head = (head + 1) % history.length;
    const { width, height } = graph;
    g.clearRect(0, 0, width, height);
    const y = (v: number) => height - (Math.min(v, GRAPH_MAX_MS) / GRAPH_MAX_MS) * height;
    g.strokeStyle = "rgba(255,255,255,0.15)"; // 60 fps budget line
    g.beginPath();
    g.moveTo(0, y(1000 / 60));
    g.lineTo(width, y(1000 / 60));
    g.stroke();
    g.strokeStyle = "#3b82f6";
    g.lineWidth = 1.5;
    g.beginPath();
    for (let k = 0; k < history.length; k++) {
      const x = (k / (history.length - 1)) * width;
      const v = history[(head + k) % history.length];
      if (k === 0) g.moveTo(x, y(v));
      else g.lineTo(x, y(v));
    }
    g.stroke();
  },
```

The last 120 frame times live in a fixed `Float32Array` used as a **ring buffer**: `head` is where the next value goes, wrapping around with `%`, so the oldest value is always overwritten and nothing is ever shifted or allocated.

Drawing starts at `head` (the oldest) and walks forward, so the line reads left to right in time. A faint line marks 16.7 ms, the budget for 60 fps: any spike above it was a dropped frame. Times are capped at 50 ms so one huge hitch doesn't flatten the rest of the graph.

CHAPTER 3

Interview questions

Q1 **Why a fixed timestep? What's the spiral of death?**

A fixed step keeps the solver stable and the simulation reproducible whatever the frame rate. Frames bank their time and spend it in whole steps; α interpolates for drawing. The spiral of death is when a slow frame leaves more steps to run, which makes the next frame slower still. Capping the steps per frame and dropping the excess breaks the loop: the simulation slows down instead of freezing.

Q2 **Explain broad phase versus narrow phase.**

The broad phase cheaply finds pairs that might touch, using bounding boxes and a spatial structure (a hash grid here, sweep and prune in cannon-es). The narrow phase does the exact test on those candidates and produces the contact: normal, depth and the data the solver needs. The split is what makes thousands of bodies affordable.

Q3 **Why does the solver iterate? What happens with too few iterations?**

Contacts are coupled: fixing one disturbs its neighbours, so a single pass leaves errors. Each sweep reduces them (Gauss–Seidel). Too few iterations and stacks feel soft: they sink, jitter, or never come to rest. More iterations cost linearly. Warm starting is the usual way to get the same stiffness with fewer passes.

Q4 **Tall stacks wouldn't settle. Why, and how did you fix it?**

Every box in the stack had the same downward velocity, so the relative velocity between neighbours was zero and the solver never applied an impulse that stopped the fall: only the bottom contact did any work. Sorting pairs bottom-up and, on the last pass, treating the lower box of each vertical contact as immovable (shock propagation) lets the floor's support travel to the top in one sweep.

Q5 **How does the conveyor move boxes when its collider never moves?**

Friction is computed from relative velocity at the contact. For the belt, the surface's velocity is the belt speed, so a box at rest is sliding backwards relative to it and friction accelerates it up to belt speed. No special conveyor code exists; jams and slipping fall out of the friction model.

Q6 **What's structure of arrays, and why is it faster in JavaScript?**

One typed array per property instead of one object per entity. Hot loops touch contiguous numbers, which caches and prefetchers love, typed arrays avoid boxing and hidden-class checks, and there are no per-entity objects for the garbage collector. It's the layout data-oriented engines and ECS frameworks use.

Q7 **What's a draw call, and what did instancing change?**

A draw call is one CPU-to-GPU request to draw some geometry with some state, each with fixed overhead. One mesh per box meant one call per box: 484 for ~500 boxes. One `InstancedMesh` draws all of them in a single call with per-instance matrices and colors: 27 calls for the whole scene.

Q8 **How do sleeping and islands work, and what went wrong?**

Bodies slower than a threshold for 0.4 s sleep and are skipped. Removing or hitting a body wakes everything transitively touching it (its island), using the spatial hash to find neighbours. Bugs found: boxes slept on a stopped belt and ignored it restarting (fixed by an on-belt flag and waking all bodies when the speed changes), and the wake stack overwrote itself while spreading (fixed by draining it properly).

Q9 **How did you benchmark fairly?**

Same seeded scene for every run, a JIT warm-up before measuring, a headless Node benchmark so rendering doesn't interfere, per-phase timing, and reporting falling and settled separately, since sleeping changes the cost by half.

CHAPTER 3

Exercises

3.1 Warm starting

Store each pair's normal impulse keyed by its two slots and apply most of it at the start of the next step. Measure how few iterations the ten-box tower test then needs.

Hint. Key pairs with `i * MAX_BODIES + j` in a Map. Clear entries for pairs that weren't found this step.

3.2 A tunnelling lab

Make the floor 1 cm thick and raise `MAX_SPEED`. Find the impact speed at which boxes start falling through, then fix it with a swept test against the floor.

Hint. A box moves v·dt per step. It tunnels when that exceeds the floor thickness plus its own height.

3.3 Sweep and prune

Implement SAP behind the same interface the spatial hash offers and compare pair counts and step time in `npm run bench`.

Hint. Keep the sorted order between steps and re-sort with insertion sort: nearly-sorted input makes it close to O(n).

3.4 A pusher

Add a moving static box that sweeps across the belt and pushes boxes off the side.

Hint. Treat it like the belt: a surface with a velocity. Its velocity is (position − previous position) ÷ dt.

3.5 Iterations versus settling

Plot (or print) the time the ten-box tower takes to fall asleep for ITERATIONS from 2 to 12, with and without shock propagation.

Hint. Make ITERATIONS a constructor option for the experiment; the tests can keep the default.

APPENDIX A

Running the projects

Every project needs Node.js 20 or newer, and Snack 1 needs 22. From `Portfolio/vention/`:

Project	Commands
`snack1-robot-arm`	`npm ci` · `npm run dev` (http://localhost:5173) · `npm test` · `npm run check` (type-check + tests) · `npm run build:gripper` regenerates the .glb · needs Node 22 (Appendix C)
`snack2-control-panel`	`npm install` · `npm run dev` · `npm test` (IK and program tests) · `npm run build`
`snack3-conveyor-physics`	`npm install` · `npm run dev` · `npm test` · `npm run bench` (headless physics benchmark)
`snack4-refactor-kata`	`npm install` · `npm test` (spec + golden master) · `npm run typecheck` · `npm run demo`
`snack5-inventory`	`docker compose --profile postgres up --build` (or `--profile mongo`), then http://localhost:8081 · `docker compose --profile test run --rm --build api-test` runs the contract tests on both databases · without Docker: `cd api; npm install; npm test`

Snack 4's full commit history ships as `history/refactor-steps.bundle`: `git clone history/refactor-steps.bundle steps`, then `git log --stat` in the new folder.

APPENDIX B

Glossary

AABB
Axis-aligned bounding box: a box whose faces line up with the world axes, described by its center and half sizes or its min and max corners.

Accessor (glTF)
Says how to read a range of a bufferView as numbers: component type, element type (VEC3, SCALAR…), count, bounds.

ACID
Atomicity, consistency, isolation, durability: the guarantees a database transaction gives.

Bin packing
Packing items into the fewest fixed-size bins; cut lists into stock bars are an instance.

Broad phase
The cheap first pass of collision detection that finds pairs which might touch.

Buffer view (glTF)
A byte range within a buffer, usable directly as a GPU buffer.

Characterization test
A test that records what code currently does, to detect changes, rather than what it should do.

CHECK constraint
A rule on a table that the database enforces on every write, rejecting rows that break it.

Conditional update
An update whose filter contains the rule (“only if enough is available”), so check and write are one atomic step.

Connection point
A named spot on a kind of object, stored as fractions of its size, where it can be joined to another.

Contract test
One test suite run against every implementation of an interface, so they behave the same.

CORS
Cross-origin resource sharing: headers by which a server lets pages from other origins read its responses in a browser.

Counting sort
A non-comparison sort that buckets items by a small integer key in O(n).

CTE
Common table expression (WITH … AS): names a sub-query; in Postgres it can also chain data-changing statements.

Deadlock
Two transactions each waiting for a lock the other holds; avoided by locking in a consistent order.

Deadlock
A state where each party waits for something only another waiting party can provide, so none can proceed.

Denavit–Hartenberg
A standard way to describe each joint of a robot arm with four parameters, giving one transform matrix per joint.

Diverter
A device on a conveyor that steers items off the line to one side, here a hinged flap.

Document database
A database storing self-contained JSON-like documents, which can embed arrays and sub-objects (MongoDB).

Draw call
One request from the CPU to the GPU to draw geometry with a given state. Each has fixed overhead.

Emissive
Light a material gives off regardless of the scene's lights; used to make the selected housing glow.

Event delegation
One listener on a container handling events from its children, identified by attributes, so re-rendered children need no new listeners.

First-fit decreasing
A bin-packing heuristic: sort items largest first, put each in the first bin with room.

Fixed time step
Advancing a simulation in equal small steps, independent of the display's frame rate, so results don't depend on it.

Foreign key
A column that must match a key in another table, so references are always valid.

Forward kinematics
Computing the tool's position from joint angles. Always one answer.

Frustum culling
Skipping objects whose bounding sphere is outside the camera's view.

Golden master
Recorded outputs of existing code for many inputs, compared against new code.

Idempotency key
A client-chosen key that makes retrying a request safe: the server returns the stored result instead of acting twice.

Impulse
An instant change in momentum, the solver's unit of correction.

Instancing
Drawing one geometry many times in one draw call, with per-instance data such as a matrix and color.

Interpolation (α)
Drawing a moment between the last two physics states, by the fraction α of a step left over.

Inverse kinematics
Computing joint angles that put the tool at a target. Zero, several or infinitely many answers.

Island
A group of bodies connected by contacts; they sleep and wake together.

Isolation level
How much concurrent transactions can see of each other (READ COMMITTED, REPEATABLE READ, SERIALIZABLE).

Jacobian
The matrix of partial derivatives of tool position with respect to joint angles; maps joint speeds to tool speed.

jsonb
PostgreSQL's binary JSON type: stores a document in a column, with operators and indexes to query inside it.

Kanban
Pull-based production: a step makes or sends an item only when the next step signals it needs one.

Kerf
The width of material a saw blade removes with each cut.

Kinematic body
A body moved by code, with a velocity, that pushes others but is never pushed itself.

Kinematic simulation
Motion set directly (positions and velocities) rather than computed from forces.

Kustomize overlay
A directory that builds on a base set of Kubernetes manifests, adding or changing resources.

Little-endian
Byte order with the least significant byte first; required by the glTF container.

Local frame
A coordinate system attached to an object, in which its own geometry is simple; points are transformed into it.

Lost update
A race where two transactions read the same value and one's write silently overwrites the other's.

moveJ / moveL
Robot moves that interpolate joint angles (curved tool path) or the tool position (straight line).

Narrow phase
The exact collision test on candidate pairs, producing contact normal and depth.

Normal
A unit vector perpendicular to a surface; drives lighting and, in physics, the direction of contact forces.

Normalization
Storing each fact once in its own table, related by keys, instead of repeating it.

Object pool
Preallocated objects or slots reused instead of created and destroyed.

ON DELETE CASCADE
A foreign-key option that deletes the referencing rows when the referenced row is deleted.

Origin
The scheme, host and port of a page (http://localhost:5173); browsers isolate pages by origin.

PBR
Physically based rendering: materials described by base color, metalness and roughness, lit consistently in any environment.

Preflight
The OPTIONS request a browser sends before a cross-origin request that isn't simple, asking whether it's allowed.

Pure function
A function whose output depends only on its inputs and which has no side effects.

Quaternion
Four numbers representing a 3D rotation, free of gimbal lock and easy to interpolate.

Raycasting
Intersecting a ray (here from the camera through the mouse) with scene geometry, for picking.

Re-parenting
Moving an object to a new parent in the scene graph; `attach` keeps its world pose, `add` keeps its local one.

Readiness probe
A Kubernetes check that decides whether a pod receives traffic; separate from liveness, which decides restarts.

Reconciliation
React's comparison of a new render's output with the previous one, to find the DOM changes to make.

Refactoring
Changing the structure of code without changing its behaviour, in small steps.

Replica set
A group of MongoDB servers holding the same data; required for multi-document transactions.

Repository pattern
Hiding storage behind an interface of domain operations, so the rest of the code doesn't know the database.

Ring buffer
A fixed array used as a queue by wrapping an index, so nothing is shifted or allocated.

Row lock
A lock on specific rows (SELECT … FOR UPDATE) that makes other writers wait until the transaction ends.

Rule engine
A small interpreter that evaluates conditions stored as data, instead of decisions written as code.

Saga
A multi-step operation made of individually atomic steps, with compensating steps to undo earlier ones on failure.

Scene graph
A tree of objects whose transforms compose from parent to child.

Seam
A place where code's behaviour can be observed or substituted without editing it, which makes testing possible.

Semi-implicit Euler
Integration that updates velocity first, then position from the new velocity; stable and cheap.

Sequential impulses
Solving contact constraints one at a time and repeating, a form of projected Gauss–Seidel.

Shock propagation
Treating the lower body of a vertical contact as immovable on a final, bottom-up pass, so stacks settle.

Singularity
A pose where the arm loses the ability to move the tool in some direction; the Jacobian loses rank.

Slop
Overlap allowed between resting bodies before position correction acts, to prevent jitter.

Smoothstep
The easing curve $3t^2 - 2t^3$: from 0 to 1 with zero slope at both ends.

Spatial hash
A uniform grid of cells stored in a hash table, for finding nearby objects in O(n).

Spiral of death
A slow frame causing more physics steps, making the next frame slower; avoided by capping steps.

Stand-in object
A temporary value built only so existing logic (here, rule matching) can be applied to something that doesn't exist yet.

StatefulSet
A Kubernetes workload with stable pod names and a persistent volume per pod, used for databases.

Structure of arrays
Storing each property of many entities in its own array, instead of one object per entity.

TCP
Tool center point: the point on the tool that motion is planned for; here, between the fingertips.

Tearing (React)
Different components showing different versions of external data in the same commit.

Tessellation
Converting exact surfaces (from CAD) into triangles for rendering.

Upsert
Insert a row, or update it if one with the same key exists, in one statement (INSERT ... ON CONFLICT DO UPDATE).

Winding order
The order a triangle's corners are listed; counter-clockwise as seen marks the front face.